IMPLEMENTATION REPORT

Centro

> AI-powered document collection platform for accounting firms — Pilot MVP build

Prepared for	noammaarachot@gmail.com
Source spec	Centro Engineering Product Specification (EPS) v2.0, Ch. 1–26
Report date	July 19, 2026
Milestones delivered	M1 – M13 (13 of 13), all committed to git
Commits	13 (1 pre-existing landing page + 12 build milestones)
Lines changed	+19,617 / −4,053 across 74 files
Stack	Next.js 16 (App Router) · TypeScript · Drizzle ORM · PostgreSQL

1. Executive Summary

Centro's Pilot MVP was built from the EPS end to end across thirteen milestones (M1–M13), each independently verified with scripted, real HTTP end-to-end tests against a live embedded Postgres database — not just type-checks. Every EPS chapter with pilot-scope requirements (Ch.1–Ch.20, plus the operational chapters 21–26) has corresponding working code, with the exception of pieces that require external credentials this environment doesn't have (Google OAuth, WhatsApp Business API, an LLM/OCR provider) — those are implemented as clearly-documented mock adapters with identical interfaces, so a real integration is a contained swap, not a redesign.

The single largest event during the build was a security audit (part of M12) that found and fixed a real, severe cross-tenant data-isolation bug: several server actions mutated resources by ID alone, without confirming the resource belonged to the caller's organization. This is detailed in full in Section 8 — it was found, fixed, and independently re-verified with a scripted cross-organization attack before the milestone was considered complete.

Milestones	13 of 13 complete
Database tables	13 (7 migrations, 0000–0006)
App routes	18 pages/routes, 2 API endpoints
Server actions	~45 across 9 action files
Scripted E2E checks run	190+ individual assertions across 13 milestone test passes, all green at time of writing
Mocked integrations	Google Drive, WhatsApp Business API, AI/OCR classification — real credentials not available in this environment
Real, non-mocked infra	Auth/sessions, audit log, state machines, scheduler, business-hours gating, resilience/retry, security fixes

> **How to read "mocked":** a mocked integration means the *transport* (the actual network call to Google/Meta/an LLM provider) is a deterministic stand-in, generating realistic IDs/responses instead of calling a real API. In every case, the *decision logic* around it — matching, validation, retries, gating, state transitions — is real, not a placeholder, and was built to the exact interface a real integration would need to implement, so swapping the transport later touches one file, not the surrounding system.

2. Table of Contents

3. System Architecture

3.1 Stack

Layer	Choice	Why
Framework	Next.js 16.2.10 (App Router, Turbopack)	Already the project's stack; AGENTS.md flagged it as having breaking changes from training-data knowledge, so the actual v16 docs (bundled in `node_modules/next/dist/docs`) were read before writing routing/data code.
Language	TypeScript, strict mode	Existing project convention.
Database	PostgreSQL via Drizzle ORM	Matches EPS DB-8.1 exactly ("PostgreSQL is the primary database").
Local dev DB	PGlite (embedded WASM Postgres)	Zero-install developer experience — `npm install && npm run dev` works with no local Postgres. Production requires a real `DATABASE_URL` (enforced at runtime).
Auth	Custom session-cookie auth, scrypt password hashing	EPS Ch.13 specifies a single shared employee account per firm — no off-the-shelf multi-user auth library fit that model.
Styling	Tailwind CSS v4, existing design tokens	Reused the landing page's design system (Ch.26) for visual consistency.
Mutations	React 19 Server Actions (progressive enhancement forms)	Idiomatic for Next.js 16; every mutation is a real HTTP-testable POST, which is how all E2E verification was done.

3.2 Module Map

Matches the EPS Ch.7 module table directly:

EPS Module	Implementation
Authentication	`src/lib/auth/` (session, password, rate limiter), `src/app/login/`
Client Management	`src/app/(app)/clients/`, `src/app/(app)/services/`
Collection Engine	`src/lib/collectionRequestStateMachine.ts`, `src/app/(app)/collections/`
Messaging	`src/lib/conversationOrchestration.ts`, `src/lib/scheduler.ts` (WhatsApp transport mocked)
Document Processing	`src/lib/ai/` (classification mocked, pipeline logic real)

Storage	`src/lib/storage/driveAdapter.ts` (Drive transport mocked)
Dashboard	`src/app/(app)/dashboard/`, `src/app/(app)/audit/`

3.3 Architectural Decisions Worth Knowing

- **Modular monolith** (AR-7.1) — one Next.js app, no microservices, matching pilot scope explicitly.
- **Mock-first integrations** — agreed with the user early: Google Drive, WhatsApp, and AI/OCR are all mocked behind adapter modules with the exact interface a real provider would need, because real credentials for any of them weren't available in this environment. See Section 9.
- **Single shared DB, org-scoped rows** — rather than one deployment per firm (which the EPS Ch.21 roadmap table literally shows for the pilot), every table carries `organization_id` and every query is scoped to it. This was an explicit decision the user confirmed, trading off against the "isolated deployment per firm" wording in favor of lower operational overhead for a pilot with few firms.
- **No public signup** — organizations are provisioned only via an internal CLI script (`npm run org:create`), never a public form. Also an explicit user decision, matching the EPS's sales-assisted tone rather than self-serve SaaS.
- **PGlite single-connection constraint** — discovered during testing: PGlite (used for local dev) does not reliably support two separate Node processes holding the same database directory open concurrently. All test mutations were routed through the running dev server's own HTTP endpoints rather than a parallel script once this was found.

4. Database & Data Model

13 tables across 7 migrations (`drizzle/0000` – `0006`), matching the EPS Ch.4/Ch.8 domain model plus the operational additions from later chapters (business-hours config, audit trail precision, Drive folder mapping).

Table	Purpose	Added
`organizations`	Tenant root. Also carries onboarding/integration status (Ch.3) and Ch.18 scheduling config (business hours/days, reminder interval, inactivity timeout).	M1, extended M4 & M8
`users`	The single shared employee account per organization (BR-13.1) — one row per org, not per person.	M2
`sessions`	DB-backed session tokens (cookie-based auth).	M2
`audit_logs`	Immutable event log (Ch.17). Insert-only. Carries `organization_id`, `client_id`, `collection_request_id` (added M11 for FR-17.3 precision), actor type/user, event type, description, optional metadata.	M2, extended M11
`clients`	Business records only, never authenticate (Ch.2). Unique `(organization_id, phone)` — one client per WhatsApp number. Carries `drive_folder_id` (M5).	M3, extended M4 & M5
`services`	Recurring document-requirement definitions (BR-001: "Services define requirements; clients do not").	M3
`service_document_requirements`	Per-service requirement templates (e.g. "Bank Statement").	M3
`client_services`	Many-to-many Client↔Service assignment.	M3
`collection_requests`	One collection cycle for one client/service/period. Ch.6 state machine status enum.	M3
`collection_request_requirements`	Per-cycle *snapshot* of the service's requirement templates (BR-002: historical requests are immutable to later service edits).	M3
`documents`	Metadata + Google Drive reference (never file bytes). Ch.6 status enum plus `deleted_from_drive` (Ch.14) with a `drive_deleted_at` timestamp.	M3, extended M5
`conversations`	One WhatsApp conversation per active Collection Request (BR-003). Ch.6 status enum including `human_control`.	M3
`messages`	Inbound/outbound message log, actor-typed sender.	M3

Every table's primary key is a UUID (DB-8.2). Every business table carries `organization_id` (DB-8.4). Foreign keys use `onDelete: "cascade"` for true parent/child ownership (e.g. a deleted org cascades everything) but deliberately `onDelete: "restrict"` (the default) for business-history references — a Client or Service with real Collection Request history cannot be hard-deleted, surfaced as a friendly UI error rather than a crash. `audit_logs` is the one exception: its client/user references use `onDelete: "set null"`, since a deleted client must never become permanently undeletable-in-spirit just because it was once mentioned in a log entry (a real bug found and fixed in M3 — see Section 5).

5. Milestone-by-Milestone Build Log

M1 — Foundations & Environment

EPS Ch.7, Ch.19

Postgres/Drizzle data layer with automatic dev/prod driver selection (`src/db/index.ts`): production requires a real `DATABASE_URL` and refuses to start without one; local dev falls back to embedded PGlite with zero setup. Env var scaffolding (`.env.example`), migration tooling (`npm run db:generate` / `db:migrate`), and the `(app)` route group established as the internal-app boundary separate from the untouched marketing site.

Bug found & fixed: Turbopack's dev bundling broke PGlite's WASM/filesystem asset loading with an unrelated-looking "path argument" TypeError on every DB call — fixed via `serverExternalPackages` in `next.config.ts` . `next build` 's static analysis never exercises this path (DB routes are `force-dynamic`), so it only surfaced under `next dev` .

M2 — Auth, Organization & Audit Log

EPS Ch.2, Ch.13, Ch.17

Shared-account login (scrypt password hashing, DB-backed sessions, `requireSession()` guarding every protected route), Organization + shared User creation (via the internal CLI provisioning script, per the no-public-signup decision), and `recordAuditEvent()` as the single insert-only write path onto the audit log.

Verified end-to-end over real HTTP: login reject/accept, protected-route redirect, logout, re-redirect after logout — including correctly decoding Next.js Server Actions' multipart wire format for scripted testing (not a trivial form POST).

M3 — Core Domain Schema + Client/Service CRUD

EPS Ch.4, Ch.8

Full schema for every remaining entity. Hand-managed CRUD only for Clients and Services (the two entities an admin actually creates directly) — Collection Requests/Documents/Conversations/Messages get their tables now but no "create" UI, since they're generated by the automation engine in later milestones, not typed in by hand.

Bug found & fixed: `audit_logs.client_id` had no `onDelete` behavior, so logging a client's own creation event (which happens immediately) made that client permanently undeletable forever after — the FK blocked it. Fixed with `onDelete: "set null"` : the log entry and its description text survive, only the structured reference clears.

M4 — Onboarding

EPS Ch.3

Mocked Google/WhatsApp connection toggles, CSV client import (hand-written parser — see the Excel note in Section 11), onboarding summary, and the BR-001 automation-activation gate enforced server-side (not just by disabling a button). `clients.phone` made unique per organization, with friendly duplicate-handling in both the CSV import and the manual client form.

M5 — Google Drive Storage (mocked)

EPS Ch.14, PR-005

`src/lib/storage/driveAdapter.ts` : `ensureClientFolder` (BR-3.003 — stores the folder ID, not its name, created lazily) and `uploadDocument`, wired to the real approval path (document approval in both the manual-add and review-document flows triggers upload the moment status becomes `approved`, per BR-11.5). Ch.14's manual-deletion reconciliation is fully implemented: a document "deleted from Drive" keeps its database row, flips status, and the UI shows "Deleted manually on DD/MM/YYYY HH:MM."

M6 — Scheduler (the real automatic trigger)

EPS Ch.5, Ch.16, Ch.18

`src/lib/scheduler.ts` + `POST /api/cron/tick` (bearer-token gated via `CRON_SECRET`, required in production): finds conversations idle past the org's configured inactivity timeout and evaluates them; finds conversations stuck waiting for a client reply past the reminder interval and sends a reminder. This is the actual automatic trigger that M8's manual "run now" buttons had stood in for.

Bug found & fixed: the evaluation function updated the Conversation's status but never the Collection Request's own status (also part of the Ch.6 state machine) — meaning the scheduler's reminder query, which correctly checks the Collection Request status, could never have matched anything. Both now transition together.

M7 — Collection Request Engine & State Machine

EPS Ch.5, Ch.6

Reordered ahead of M5/M6 mid-build (user-approved) once it became clear neither Drive nor WhatsApp integration had a real caller yet. The FR-6.2 transition graph (`draft → active → waiting_for_client → processing → completed / escalated / cancelled` , with completion reachable only via `processing` , and reopening the one deliberate exception back to `active`), plus the completion gate (BR-6.2: processing documents block completion; BR-11.2/BR-6.1: every requirement needs an approved document). Manual document marking (BR-11.3) lets an employee attach a document with a status directly — a real, spec-anticipated capability independent of the AI pipeline, not a placeholder for it.

M8 — Conversation Orchestration

EPS Ch.10, Ch.16, Ch.18

`src/lib/conversationOrchestration.ts` is the WhatsApp transport boundary — one function (`sendOutboundMessage`) is the single call site a real Business API integration replaces. Real logic: `ensureConversation` (BR-003), the initial-request send, identical-filename duplicate detection (later superseded by M9's fuzzier version), `evaluateAndPrompt` (the "after N minutes of inactivity" decision, silent when unsatisfied), and `reopenIfCompleted` (a new document after completion reopens the request automatically). Since there was no scheduler yet at this point, the inactivity timer and inbound WhatsApp webhook were both stood in for by explicit buttons on the Collection Request page — M6 later supplied the real automatic trigger for the same functions.

M9 — AI Document Intelligence (mocked)

EPS Ch.9, Ch.11

`src/lib/ai/intentClassifier.ts` and `documentClassifier.ts`: no OCR/LLM provider is configured, so both use deterministic rules (keyword matching for intent; filename-token overlap against candidate requirement names for document matching) behind the exact interface a real provider would implement. The full Ch.11 pipeline is wired for real: unsupported file types and unreadable files are rejected/flagged automatically before ever creating a document row; a confident match (≥60%) auto-approves and uploads to Drive; anything less certain — or unmatched entirely — is still filed but lands in `needs_review` for a person to resolve.

Gap found & fixed: a document classification couldn't confidently match had nowhere to render at all — the page only ever showed documents grouped under a requirement. Added an "Unmatched documents" section with a manual assign-to-requirement action.

M10 — Operational Dashboard

EPS Ch.12

Five status cards (Active, Waiting for Client, Needs Employee Review, Documents in Processing, Completed Today) each with count and percentage of all active collections (FR-12.2). "Needs Employee Review" is a proper union (not a naive sum) of escalated requests and requests with a `needs_review` document, so a request matching both criteria only counts once. Drill-down per card (FR-12.3) shows client, progress %, missing documents, last activity, status — not a full client list. Client search by name or phone (FR-12.5).

M11 — Audit Trail UI + Resilience

EPS Ch.15, Ch.17

The audit log itself existed since M2 but nothing ever displayed it — closed with an org-wide `/audit` feed and a per-Collection-Request "Audit History" section (FR-17.3), which required adding `collection_request_id` to the schema since `client_id` alone wasn't precise enough (one client can have many requests over time). `src/lib/resilience.ts` (`withRetry`): exponential-backoff retry wrapper wired into the Drive upload call site; callers catch failures and degrade gracefully (log + leave the document approved-but-unfiled) rather than crashing the action or corrupting the Collection Request (BR-15.1).

M12 — Security Hardening

EPS Ch.13, NFR-24.4

See Section 8 for full detail. Found and fixed a severe cross-tenant IDOR affecting document approval and client/service CRUD; fixed a related authorization gap in the scheduler's manual trigger; added login rate limiting (5 attempts / 15 minutes) and standard security response headers.

M13 — Full Pilot Acceptance Validation

EPS Ch.20

One continuous 32-assertion scripted run through the entire pilot lifecycle in a single organization — onboarding through business-hours config, service/requirement setup, CSV client import, collection request creation, conversation initiation, AI document classification (both auto-approved and needs-review paths), dashboard queue visibility, scheduler-driven prompting, client-confirmed completion, Drive deletion reconciliation, request reopening, and full audit trail verification. All 32 checks passed. Confirmed via `git diff` against the initial commit that the marketing landing page has zero changes across the entire build.

6. Routes, Pages & Server Actions Reference

6.1 Pages

Route	Purpose	Auth
`/`	Marketing landing page (pre-existing, untouched)	Public
`/login`	Shared-account login	Public (redirects away if already authenticated)
`/dashboard`	Work-queue status cards, drill-down, client search	Protected
`/onboarding`	Integration toggles, CSV import, automation activation	Protected
`/clients`, `/clients/new`, `/clients/[id]`	Client CRUD, service assignment, collection-request creation	Protected
`/services`, `/services/new`, `/services/[id]`	Service CRUD, requirement templates	Protected
`/collections`, `/collections/[id]`	Collection Request list/detail — status transitions, requirements, documents, conversation timeline, audit history	Protected
`/audit`	Org-wide audit log	Protected
`/settings`	Business-hours/scheduling config, manual scheduler trigger	Protected

6.2 API Routes

Route	Method	Purpose
`/api/health`	GET	DB connectivity check (ops monitoring)
`/api/cron/tick`	POST	External-scheduler-callable endpoint running the real automatic evaluation/reminder trigger. Bearer-token gated by `CRON_SECRET`.

6.3 Server Action Files

File	Actions
`src/app/login/actions.ts`	`login` (rate-limited)
`src/app/(app)/actions.ts`	`logout`

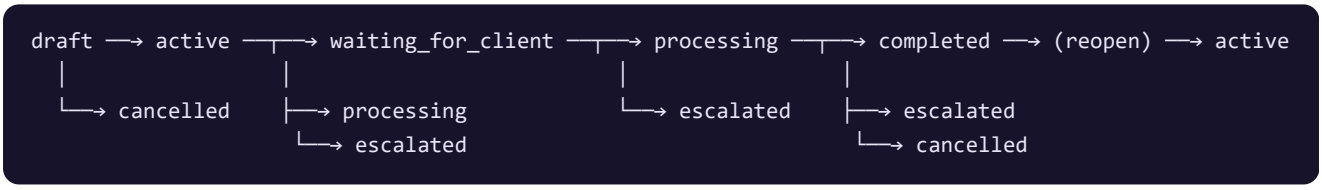
`src/app/(app)/clients/actions.ts`	`createClient`, `updateClient`, `deleteClient`, `assignService`, `unassignService`
`src/app/(app)/services/actions.ts`	`createService`, `updateService`, `deleteService`, `addRequirement`, `removeRequirement`
`src/app/(app)/collections/actions.ts`	`createCollectionRequest`, `transitionStatus`, `addManualDocument`, `reviewDocument`, `assignDocumentRequirement`, `simulateDriveDeletion`
`src/app/(app)/collections/conversationActions.ts`	`initiateConversation`, `simulateInboundMessage`, `evaluateNow`, `markFinished`, `markMoreDocuments`, `takeOverConversation`, `releaseConversation`, `sendEmployeeMessage`
`src/app/(app)/onboarding/actions.ts`	`connectGoogle` / `disconnectGoogle`, `connectWhatsapp` / `disconnectWhatsapp`, `activateAutomation`, `deactivateAutomation`, `importClients`
`src/app/(app)/settings/actions.ts`	`updateBusinessHours`, `runSchedulerNow`

7. Core Workflows

7.1 Document Collection Lifecycle (Ch.5/Ch.6/Ch.10)

1. Employee assigns a Service to a Client, then starts a Collection Request for a period (or, once a scheduler-driven auto-create trigger is wired in a future milestone, this happens on its own).
2. The Service's requirement templates are snapshotted into the request (BR-002) — later edits to the Service template never retroactively change this request.
3. Employee (or, once real WhatsApp is connected, the automated flow) initiates the conversation — the first outbound message.
4. Documents arrive (simulated inbound messages today); each runs through the Ch.11 pipeline — validated, classified, matched, auto-approved or flagged for review, stored to (mock) Drive.
5. Once all requirements are satisfied, the scheduler (or an idle-triggered evaluation) prompts the client to confirm Finished / More Documents — never assumed automatically (BR-16.1).
6. A Finished reply walks the request through `processing → completed`; a new document after completion reopens it automatically.

7.2 Collection Request State Machine



8. Security Review & the Cross-Tenant Fix

CRITICAL — FOUND AND FIXED

As part of M12, a dedicated audit was run specifically to check EPS NFR-24.4 ("each accounting firm shall operate in a fully isolated environment"): every database read/write reachable from a route param or bound server-action argument was checked for whether it verified the resource belonged to the caller's `organization_id`.

What was found

Most mutations that took a resource ID as a bound server-action argument fetched or mutated the row by ID *alone*, without checking organization ownership:

- `clients/actions.ts`: `updateClient`, `deleteClient`, `assignService`, `unassignService`
- `services/actions.ts`: `updateService`, `deleteService`, `addRequirement`, `removeRequirement`
- `collections/actions.ts`: `reviewDocument`, `assignDocumentRequirement`, `simulateDriveDeletion`, `addManualDocument`

The concrete exploit: an authenticated employee of Organization A could view, edit, approve, reject, or delete Organization B's clients, services, and documents by substituting a foreign ID into an otherwise-legitimate request — the most severe form being direct cross-tenant approval/rejection of another firm's client documents.

A related, separate issue: the `/settings` "run scheduler now" button called the exact same function the global cron endpoint uses, so any employee's click would trigger message sends and evaluations across *every* organization in the database, not just their own.

What was fixed

Every affected query now scopes explicitly to `organization_id = session.organizationId`, either directly or via shared helpers (`getOrgScopedCollectionRequest`, `getScopedDocument`, `getOrgScopedService`) that also verify parent-child ownership — e.g. a document must belong to the *specific* collection request it's addressed through, not just exist somewhere in the org. `createCollectionRequest` additionally verifies both the client and service belong to the caller's org before trusting a `client_services` assignment row. `runScheduledTasks` now takes an optional `organizationId` to scope a single run; only the cron endpoint omits it.

How it was verified

1. Two real, separate organizations were provisioned. Direct-URL access from Org B to Org A's client/collection-request/service by ID correctly returned 404 in every case.
2. A genuine wire-level forgery: Org B's own legitimately-issued server-action reference (a real, validly-signed action ID) was submitted with the UUID *inside* the bound-argument payload swapped for Org A's client ID — not just a different URL, which Next.js Server Actions don't route by. The forged delete request returned normally (200/303) but affected zero rows; Org A's client was confirmed to still exist immediately after.

3. A full regression pass of every normal same-org CRUD/assignment/collection-request/document-approval flow (15 assertions) confirmed the added checks don't break legitimate access.

Additional hardening in the same milestone

Control	Detail
Login rate limiting	5 failed attempts per 15-minute window locks out that email, even against a subsequently-correct password. Process-local (acceptable for a single-instance pilot per BR-13.2).
Security headers	`X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy`, `Strict-Transport-Security` — set on every response via `next.config.ts`.
Secrets audit	Confirmed no hardcoded credentials anywhere in source; only `.env.example` is tracked in git.
Password storage	Node's native `scrypt` (salted, timing-safe comparison) — no external dependency needed.

9. Integrations — Mocked vs. Real

Integration	Status	Real interface implemented?	What's needed for real
Google Drive	MOCKED	Yes — `ensureClientFolder` / `uploadDocument` in `driveAdapter.ts`	Google Cloud project, OAuth client credentials, Drive API scope grant
WhatsApp Business API	MOCKED	Yes — `sendOutboundMessage` in `conversationOrchestration.ts` is the single call site	Meta Business verification (has its own real-world lead time), WhatsApp Business API access, webhook endpoint for inbound messages
AI/OCR document classification	MOCKED	Yes — `classifyDocument` / `classifyIntent` in `src/lib/ai/`	An LLM API key (e.g. Anthropic) plus an OCR provider for image-based documents
Authentication	REAL	—	Nothing — fully implemented, not a stand-in
Audit logging	REAL	—	Nothing
Scheduler / cron trigger	REAL	—	An actual external cron caller (Vercel Cron, GitHub Actions scheduled workflow, etc.) needs to be pointed at `POST /api/cron/tick` — the endpoint itself is production-ready
State machines, business rules, resilience/retry	REAL	—	Nothing

> The mocking pattern is consistent throughout: each mock lives behind a small adapter module exposing the same function signatures a real integration would need, and every caller only ever touches the returned ID/result — never anything mock-specific. Swapping in a real provider is a change contained to that one file per integration.

10. Testing Performed

Every milestone was verified with a scripted, real end-to-end HTTP test — not just `npm run build` passing. Each test logs into a live dev server, drives the actual UI/forms via genuine POST requests (including correctly decoding Next.js Server Actions' multipart bound-argument wire format), and asserts on the resulting page content and/or direct database state. No test was declared passing without being run and observed green.

Milestone	What was tested	Assertions
M1	DB connectivity, migration tooling, build correctness	Direct connection + query round-trip
M2	Login/logout, session cookie behavior, protected-route redirects, audit trail accuracy	Full HTTP flow, cookie inspection
M3	Client/Service CRUD, assignment, deletion blocking, audit FK fix	11 checks
M4	Onboarding gate, CSV import (valid/invalid/duplicate rows), duplicate-phone rejection	15 checks
M5	Drive upload on approval, Drive-link display, deletion reconciliation	8 checks
M6	Scheduler evaluation branch, reminder branch, CRON_SECRET auth (401/200)	9 checks + auth verification
M7	Full state machine walk, invalid-transition blocking, completion gate, reopen	18 checks
M8	Conversation flow end-to-end, duplicate detection, human takeover/release	25 checks
M9	Unsupported/unreadable rejection, auto-approval, fuzzy duplicates, unmatched-document handling	13 checks
M10	Dashboard queue accuracy, search, queue transitions on status change	10 checks
M11	Org-wide and per-request audit history, cross-request isolation, resilience utility (4 unit-style checks)	12 checks
M12	Cross-org attack simulation (2 real orgs, forged bound arguments), rate limiting, security headers, full same-org regression	~20 checks
M13	Single continuous full-lifecycle run spanning every milestone's feature in one organization	32 checks

Total: 190+ individual scripted assertions across all milestones, all passing at the time of writing.

> **Test harness note:** several genuine test-script bugs were found and fixed along the way (ambiguous form-marker matching when a page has multiple similar forms; React's SSR comment-marker text interpolation breaking naive substring checks; forgetting to set a required form field). Each is called out in the relevant milestone's commit message where it could be mistaken for an app bug. Two real *application* bugs were also

found this way and are documented in Section 5 (the audit-log FK issue in M3, the scheduler status-sync issue in M6) alongside the Section 8 security finding.

11. Known Limitations & Deferred Scope

Item	Why
Excel (.xlsx) import not implemented — CSV only	The only npm-available parser (`xlsx` /SheetJS) carries unpatched high-severity vulnerabilities (prototype pollution, ReDoS) since SheetJS moved patched releases off the public registry. CSV covers the same functional need (FR-005) without that risk; revisit if a trustworthy parser becomes available.
Google Drive, WhatsApp, AI/OCR are mocked	No credentials available in this environment. See Section 9 for exactly what's needed to make each real.
Rate limiting is process-local (in-memory)	Sufficient for a single-instance pilot (BR-13.2); would need a shared store (Redis, etc.) if ever deployed as multiple instances.
No automatic scheduled Collection Request creation	Ch.5's "automatic" trigger for *starting* a new cycle each period wasn't built — Collection Requests are created manually (or would need a small addition to the scheduler once business-hours/frequency config per-service is defined). Everything *after* creation (conversation, documents, completion, reminders) is fully automatic.
"Documents in Processing" dashboard card typically reads zero	The mocked AI classification resolves synchronously, so there's never a persisted "processing" window to observe. Wired correctly; will show real data once/if an async OCR job queue exists.
Production deployment not executed	No hosting credentials provided; the app is deployment-ready (env var separation, health check, migration scripts, CRON_SECRET-gated endpoint) but was not pushed to a live host as part of this build.

12. Files Created / Changed

74 files changed since the initial landing-page commit (+19,617 / −4,053 lines). Organized by area:

Database

`src/db/schema.ts`, `src/db/index.ts`, `drizzle/0000` – `0006` (7 migrations + snapshots), `drizzle.config.ts`, `scripts/migrate.ts`, `scripts/create-organization.ts`

Auth & Security

`src/lib/auth/session.ts`, `password.ts`, `rateLimiter.ts`, `src/app/login/` (page, form, actions)

Domain Logic

`src/lib/collectionRequestStateMachine.ts`, `conversationOrchestration.ts`, `scheduler.ts`, `businessHours.ts`, `resilience.ts`, `audit.ts`, `csv.ts`

AI (mocked)

`src/lib/ai/intentClassifier.ts`, `documentClassifier.ts`

Storage (mocked)

`src/lib/storage/driveAdapter.ts`

Data Access Layer

`src/lib/data/clients.ts`, `services.ts`, `collectionRequests.ts`, `dashboard.ts`, `auditLog.ts`, `organizations.ts`

App Routes & UI

`src/app/(app)/` — `layout.tsx`, `actions.ts`, and full subtrees for `dashboard/`, `onboarding/`, `clients/`, `services/`, `collections/`, `audit/`, `settings/`; `src/app/api/health/`, `src/app/api/cron/tick/`

Config

`next.config.ts`, `.env.example`, `package.json` (scripts)

Untouched (verified via git diff)

`src/app/page.tsx`, `src/app/layout.tsx`, all of `src/components/landing/`

13. Recommended Next Steps

1. **Decide on real credentials** for Google Drive, WhatsApp Business API, and an AI/OCR provider — each swap is contained to one adapter file (Section 9), but Meta's Business verification in particular has its own lead time worth starting early.
2. **Choose a hosting target** and provision a real managed Postgres instance; the app already enforces this split (refuses to start in production without `DATABASE_URL`) and ships a health-check endpoint for platform monitoring.
3. **Wire the external cron** (Vercel Cron, GitHub Actions scheduled workflow, or equivalent) to `POST /api/cron/tick` with the `CRON_SECRET` bearer token.
4. **Consider Excel import** once a trustworthy, actively-maintained parser is available, or if CSV-only proves insufficient for pilot firms in practice.
5. **Run the pilot** against Ch.20's success metrics (SM-20.1–20.4) with a real accounting firm once the above integrations are live — the dashboard, audit trail, and state machine are all in place to observe those metrics directly.

ADDENDUM

Centro Implementation Report

Epic 2 — Authentication & Onboarding Preparation

This addendum extends the Centro Implementation Report (Milestones M1–M13, dated July 19, 2026) with the work delivered in Epic 2. It is bound directly after the original report and should be read together with it. Two statements in the original report's Section 3.3 are superseded by this epic — see Section 14.2 below.

Prepared for	noammaarachot@gmail.com
Addendum covers	EPIC 2 — Authentication & Onboarding Planning (Feature 1, 2, 3)
Addendum date	July 19, 2026
Applies on top of	Centro Implementation Report, Milestones M1–M13 (bound below)
Commit	`feat(auth): registration flow and onboarding preparation`

14. Epic 2 — Authentication & Onboarding

14.1 Executive Summary

Epic 2 added self-service account registration, separated the landing page's lead-capture popup from the authenticated application, and put in place (without yet building the full wizard) the architecture for a first-time-onboarding gate. No visual/design-system change was made — every new form and page reuses the existing `Card` / `Button` / `TextField` primitives and copy tone introduced in the prior UI/UX redesign.

NEW DB COLUMNS	NEW ROUTES	RELOCATED ROUTE	NEW SERVER ACTIONS
4 (2 tables)	**/terms, /privacy**	**/onboarding**	**register, finishOnboarding**

MIGRATION
drizzle/0007

14.2 Superseded Decisions from the Original Report

Corrections to Section 3.3

- **"No public signup"** — the original report states organizations were provisioned only via the internal CLI script (`npm run org:create`), matching a sales-assisted, no-self-serve model. Epic 2 explicitly reverses this: `/login` now offers a "Create Account" tab that self-registers a brand-new organization. The CLI script still exists and still works (internal/pilot provisioning), but is no longer the only path.
- **Onboarding location** — Section 6.1 lists `/onboarding` as a Sidebar-accessible page inside the authenticated app shell, always reachable. It has been relocated outside that shell (no longer in the Sidebar nav) and is now a first-login-only gated flow — see 14.6.

14.3 Database & Data Model Changes

Migration `drizzle/0007_gray_mentallo.sql` , applied via the existing `npm run db:migrate` tooling (`src/db/index.ts` , unchanged). All four new columns are nullable — no backward-incompatible schema change.

Table	Column	Purpose
`organizations`	`onboarding_completed_at`	Nullable timestamp. Same pattern as the existing `googleConnectedAt` / `whatsappConnectedAt` / `automationActivatedAt` columns. Drives the onboarding routing gate (14.6).

`users`	`full_name`	Nullable — CLI-provisioned accounts still have none. Always set by self-service registration.
`users`	`terms_accepted_at`	Nullable. Consent timestamp, set at registration.
`users`	`privacy_accepted_at`	Nullable. Consent timestamp, set at registration.

Backfill: the same migration sets `onboarding_completed_at = COALESCE(automation_activated_at, now())` for every pre-existing organization, so pilot orgs created before Epic 2 are not sent back into the onboarding gate on their next login — only genuinely new organizations start ungated.

14.4 Registration Flow

`/login` now renders a tab switcher (`src/app/login/AuthTabs.tsx`) between the existing Login form and a new Create Account form (`src/app/login/RegisterForm.tsx`) inside the same card shell — no new route, so every existing `redirect("/login")` call across the codebase kept working unchanged.

- Fields: Full Name, Email, Password, Confirm Password — with show/hide toggles on both password fields.
- Two required checkboxes (Terms & Conditions, Privacy Policy), linking to new minimal placeholder pages (`/terms` , `/privacy` — the landing Footer's legal links were previously `href="#"`).
- Server-side validation (`register()` in `src/app/login/actions.ts`): required fields, email format and uniqueness, password minimum length (8), password/confirm match, both checkboxes accepted.
- On success: inserts a new `organizations` row and a new `users` row (password hashed with the existing scrypt-based `hashPassword()`), records two audit events (`organization.created` , `employee.registered` , reusing `src/lib/audit.ts`), creates a session (`createSession()` , existing helper — the user is logged in immediately, never sent back to the login screen), then routes through the shared post-auth gate (14.6).

> **The organization name is a temporary placeholder, not a product decision**
>
> Registration deliberately does not ask for an office/company name — only Full Name, Email, Password, Confirm Password, per spec. Since `organizations.name` is `NOT NULL` , it is filled with the registrant's full name purely to satisfy that constraint. The real office name is planned as onboarding-wizard step 1 in the next epic; nothing in this epic treats the placeholder as final, and no UI surface introduced in Epic 2 displays it as "your organization name."

14.5 Login Flow (Updated)

`login()` is otherwise unchanged (same rate limiting, same credential check, same audit event) — only its final step changed: instead of a hardcoded `redirect("/dashboard")` , it now calls the same shared gate function registration uses (14.6), so both entry points make the exact same onboarding-vs-dashboard decision through one function, not two separately-maintained redirects.

14.6 First-Time User / Onboarding Architecture

New module `src/lib/onboarding.ts` is the single source of truth for the onboarding gate:

Export	Role
`needsOnboarding(organization)`	The one predicate: `!organization.onboardingCompletedAt`.
`redirectAfterAuth(organizationId)`	Loads the organization, applies `needsOnboarding()`, and redirects to `/onboarding` or `/dashboard`. Called as the last step of both `login()` and `register()`.
`markOnboardingComplete(organizationId)`	Sets `onboardingCompletedAt = now()`. Called from the "continue to dashboard" action and from `activateAutomation()` — the seam the real onboarding wizard (next epic) will also call from its own final step.

`src/app/(app)/layout.tsx` — the shared layout every Dashboard/Clients/Services/Collections/Settings/Audit route renders through — also calls `needsOnboarding()` as a defensive backstop (e.g. a bookmarked `/dashboard` URL from before onboarding finished), reusing the same predicate rather than re-implementing the condition anywhere else.

What "Office Setup" is today, and what it becomes: the page previously reachable from the Sidebar as "הקמת המערכת" (Google/WhatsApp connect toggles, CSV client import, automation activation — all built in M4 of the original report) was moved, content and logic untouched, from `src/app/(app)/onboarding/` to `src/app/onboarding/` (same URL — Next.js route groups don't affect the path, so no `redirect()` string anywhere needed to change). It's no longer a Sidebar nav item; it is now what a first-time user lands on immediately after registering or logging in, and it's the placeholder the real onboarding wizard replaces in the next epic. One new action, `finishOnboarding()`, adds an explicit "Continue to Dashboard" exit so an organization is never stuck there just because it isn't ready to activate automation yet.

14.7 Landing Page / Application Separation

Root cause: `DemoRequestModal` (the 45-second lead-capture popup) was mounted in `src/app/layout.tsx` — the root layout every route in the app renders through, authenticated or not — so its internal timer could fire on `/login`, `/onboarding`, or any Dashboard-family route.

Fix: moved the `<DemoRequestModal />` mount from the root layout into `src/app/page.tsx` (the actual public landing page) alongside its other sections. It is no longer part of the module graph for any other route, so it cannot mount there. `AccessibilityProvider` / `AccessibilityWidget` / `MotionProvider` are unrelated to this popup and were left in the root layout unchanged.

14.8 Routes & Server Actions — Delta on Section 6

Route	Change
`/login`	Now renders both Login and Create Account (tabbed); redirects an already-authenticated visitor through the shared gate instead of straight to `/dashboard`.

`/onboarding`	Moved out of the `(app)` route group (no Sidebar); gated to first-time users instead of always reachable.
`/terms`, `/privacy`	*New.* Minimal placeholder pages linked from the registration checkboxes.

File	Actions added
`src/app/login/actions.ts`	`register` (new); `login` now ends with `redirectAfterAuth()` instead of a hardcoded redirect.
`src/app/onboarding/actions.ts`	`finishOnboarding` (new); `activateAutomation` now also calls `markOnboardingComplete()`.

14.9 Testing Performed

Same real-HTTP, scripted-E2E standard as M1–M13: a Playwright script drove an actual browser against the running dev server (temporarily installed for the test run, removed afterward — not a project dependency). All of the following were observed passing, not just asserted:

- Register a new account → lands on `/onboarding` directly (not `/dashboard` first) → Sidebar absent on that page → "Continue to Dashboard" → lands on `/dashboard`.
- Log out, log back in with the same (now-onboarded) account → lands straight on `/dashboard`, confirming the gate persists correctly across sessions.
- Log in with a pre-Epic-2 account (created before this migration) → lands straight on `/dashboard`, confirming the backfill in 14.3.
- Duplicate-email registration and password-mismatch submissions both surface the correct field-level error and do not create a row.
- 46-second real-time wait on an authenticated `/dashboard` session confirmed the demo popup never appears.
- `npm run build` and `npm run lint` both clean.

14.10 Known Limitations & Next Steps (Epic 2)

Item	Why / Next step
Onboarding wizard not built	Explicitly out of scope for this epic — Feature 3 asked only for the routing/gating architecture to be ready. The relocated Office Setup page is the interim placeholder; `markOnboardingComplete()` is the seam the real wizard calls next.
Organization has no rename UI yet	The placeholder name set at registration (14.4) can't currently be edited anywhere in the app (Settings only covers business hours). Real name collection is planned as the wizard's first step.

`/terms` and `/privacy` are placeholders	No legal copy was authored as part of this epic — the pages exist so the required checkboxes link to something real, not to publish final legal text.
CLI provisioning script unchanged	`scripts/create-organization.ts` still works as before for internal/pilot use and does not collect a full name — those accounts keep `users.full_name = NULL`.

Epic 2 addendum — generated from the actual repository state (git history, schema, and a live scripted test pass) at completion of the Authentication & Onboarding Planning epic. All test results described were executed and observed, not projected.

ADDENDUM

Centro Implementation Report

Epic 3 — Premium First-Time Onboarding Wizard

This addendum extends the Centro Implementation Report (Milestones M1–M13) and the Epic 2 addendum (Authentication & Onboarding Preparation) with the work delivered in Epic 3. It is bound directly after both and should be read together with them.

Prepared for	noammaarachot@gmail.com
Addendum covers	EPIC 3 — Premium First-Time Onboarding Wizard (9 steps)
Addendum date	July 19, 2026
Applies on top of	M1–M13 report + Epic 2 addendum (bound before this section)
Commit	`feat(onboarding): premium multi-step onboarding wizard with business-type classification`

15. Epic 3 — Premium First-Time Onboarding Wizard

15.1 Executive Summary

Epic 3 replaces the interim "Office Setup" page (built in Epic 2 as an explicit placeholder) with a real 9-step onboarding wizard at `/onboarding`: Welcome, Office Information, Connect Services, Import Clients, AI Client Analysis, Configure Required Documents, Reminder Rules, Summary, and Completion. It introduces a new product concept, **Business Type**, and a mocked AI classifier that sorts imported clients into it automatically — while deliberately reusing the existing Service / Collection-Request engine underneath rather than building a second, parallel system.

WIZARD STEPS	NEW DB OBJECTS	NEW STARTER BUSINESS TYPES	MIGRATION
9	**1 table, 7 columns**	**5**	**drizzle/0008**

REAL BUGS FOUND & FIXED
2

15.2 Architectural Decision: Business Type = a Product Concept, Not a New Domain Model

The central design question was how "Business Type" (Step 5's classification, Step 6's document lists, Step 7's reminder rules) relates to the pre-existing `services` / `service_document_requirements` engine, which already models "a named thing that owns a document-requirement list." The chosen approach, confirmed with the user before implementation: **Business Type is a UI/product concept only** — every business type is backed by exactly one auto-managed Service of the same name. Users never see the word "Service" anywhere in the wizard; the existing `/services` pages (untouched by this epic) continue to work as the already-built editor for those same rows. This reused 100% of the Collection Request engine, the requirement-snapshot logic, and the dashboard/audit trail, with zero new parallel machinery.

15.3 Database & Data Model Changes

Migration `drizzle/0008_cold_bucky.sql`. All additions nullable/backward compatible.

Table	Column(s)	Purpose
`business_types` *(new table)*	`id, organizationId, name, serviceId, isCustom, createdAt, updatedAt`	Org-scoped catalog (never a shared global list, so each firm can freely rename/add/ remove its own types). `serviceId` is the 1:1 link to the backing Service. Seeded with five starter types (Limited Company, Sole Proprietor, Exempt Business, Nonprofit, Payroll Only) the first time an org reaches Step 5 — the

		starter list itself is a code-level default, like any product's recommended template, but every row it produces is a real, independently-editable DB row from that point on.
`clients`	`business_type_id`	Nullable FK. Null = "Unclassified." Set by the mocked AI classifier or manually via Step 5's bulk-assign flow.
`services`	`reminder_interval_days_override`, `inactivity_timeout_minutes_override`, `business_hours_start_override`, `business_hours_end_override`, `business_days_override`	Five nullable per-Business-Type overrides mirroring `organizations`' equivalent columns exactly. Null means "use the organization's default." Every pre-Epic-3 service has all five null.
`organizations`	`logo_url`, `onboarding_step`	`logo_url`: a small logo stored directly as a data: URL (no blob storage exists in this project — a real, working solution at this size, not a mock). `onboarding_step`: which step (1–9) a first-time user last reached, so closing the tab mid-wizard resumes there instead of restarting.

15.4 Reminder Rules Are Real Per-Business-Type Overrides

Confirmed with the user as a deliberate, real (not cosmetic) change: reminder interval, inactivity timeout, and business hours/days can now differ per Business Type, not just per organization. `src/lib/businessHours.ts` gains `resolveScheduleConfig(organization, service)` — the single place that resolves the effective five-field config (service override ?? organization default). `src/lib/conversationOrchestration.ts`'s `sendOutboundMessage` (the one gating point for every automated send) now resolves the sending conversation's Collection Request → Service before checking business hours, instead of reading the organization alone. `src/lib/scheduler.ts`'s two queries (idle-open, stale-waiting) now join through to `services` and resolve each conversation's effective cutoff individually in application code, rather than computing one shared SQL cutoff per organization. Both changes are additive: a service with no overrides resolves to exactly the pre-Epic-3 organization-wide values, so the M1–M13 scheduler/ conversation test coverage's expected behavior is unchanged for all pre-existing data.

15.5 Mocked AI: Client Classification

`src/lib/ai/businessTypeClassifier.ts` follows the same "real interface, mocked transport" pattern already established by `src/lib/ai/documentClassifier.ts` (M9): no LLM credential is configured for this pilot, so classification is deterministic — an explicit business-type column in the imported file (real, office-provided data) takes priority when present; otherwise a small set of Hebrew/English legal-suffix heuristics apply

(`בע"מ`/"Ltd" → Limited Company, `עמותה` → Nonprofit, `עוסק פטור` → Exempt Business, etc.); anything unmatched is left Unclassified rather than guessed. Swapping in a real LLM later touches only this file.

15.6 Import: "Excel / CSV" Behind an Adapter

M4 deliberately shipped CSV-only because the only npm .xlsx parser (xlsx/SheetJS) carries unpatched high-severity vulnerabilities on the public registry. Per explicit direction for this epic, the wizard's Step 4 still always labels itself "Import Excel / CSV" and accepts `.csv/.xlsx/.xls` by file picker, but the actual parsing is routed through a new `src/lib/import/clientImportAdapter.ts`: `.csv` is fully real (reuses the existing hand-written parser unchanged); `.xlsx` / `.xls` are recognized and rejected with a friendly "export as CSV" message instead of a crash. Native Excel parsing — if a trustworthy package becomes available — is a change contained to this one adapter file; nothing in the wizard, its actions, or its UI needs to change.

15.7 The Wizard's Architecture

`/onboarding` is **searchParam-driven** (`?step=1` … `9`), the same pattern already used elsewhere in this app (e.g. the Dashboard's `?queue=` drill-down) — not a client-side single-page wizard with its own state machine. Each step is a focused Server Component in `src/app/onboarding/steps/`, rendered by a `page.tsx` switch statement that loads exactly the data that step needs; a shared `WizardShell.tsx` provides the progress bar, title/description, a reusable `HelpTip` "Why is this important?" popover (`src/components/app/HelpTip.tsx`, available to any future feature, not onboarding-only), and the Previous link. Inserting a new step in a future epic is a one-file-plus-one-switch-case change, not a rewrite — the explicit "modular, extensible" requirement for this epic.

Every step persists through the existing server-action convention (plain `<form action={...}>`, no client-side wizard state to keep in sync) — `src/app/onboarding/actions.ts` keeps Epic 2's `connectGoogle` / `connectWhatsapp` / `finishOnboarding` etc. entirely unchanged in behavior and reuses them directly (Step 3, Step 9); Step 6 reuses the existing `addRequirement` / `removeRequirement` actions from `services/actions.ts` unmodified in logic (only extended with an optional `returnTo` field so the wizard can reuse them without navigating away to `/services/[id]`).

Real bug found & fixed #1 — CSV parser and Hebrew legal suffixes

Testing with a realistic Hebrew client-name fixture (e.g. a company named with the extremely common `בע"מ` suffix) surfaced a real, pre-existing parsing bug in `src/lib/csv.ts` (written in M4): the hand-written parser toggled "inside a quoted field" on *any* `"` character, including one appearing mid-field in an otherwise-unquoted value — exactly what `בע"מ` contains. This silently corrupted the rest of the file into one swallowed field, losing most or all rows. Fixed to match RFC 4180 correctly: a `"` only starts a quoted field when it is the field's very first character. This is a real fix to existing import logic, not new code path — every future CSV import (wizard or otherwise) benefits.

Real bug found & fixed #2 — same-page Server Action redirects

Several wizard steps (and, once discovered, two pre-existing forms — Settings' business-hours form and `/services/[id]`'s requirement add/remove) submit a Server Action that redirects back to the exact same page it was submitted from. Next.js's client router does not reliably treat a `redirect()` to an identical URL as "new data available," so the mutation was persisted correctly to the database but the visible page kept showing pre-mutation data until a manual reload. Fixed using Next.js 16's `refresh()` API (`next/cache` — see `node_modules/next/dist/docs/.../functions/refresh.md`), which exists specifically for this case: same-page mutations now call `refresh()` and simply return, with no `redirect()` at all (the two aren't meant to compose in the same call); genuine cross-page navigations were left as plain `redirect()` calls, unaffected. Verified with a real scripted browser test asserting the updated content is visible without a manual reload, not just that the database row changed.

15.8 Step-by-Step Reference

Step	What it does	Persists via
1. Welcome	Greeting, estimated time, no data entry.	`advanceOnboardingStep(2)`
2. Office Information	Office name (replaces the Epic-2 registration placeholder) + optional logo (client-side canvas resize → data URL).	`updateOfficeInfo` — shared with Settings, see 15.9
3. Connect Services	Google Drive / WhatsApp Business toggles with plain-language "why" copy. Not a hard gate — BR-001's real activation gate still lives at completion (15.9).	Epic 2's existing `connectGoogle` / `connectWhatsapp`
4. Import Clients	Excel/CSV upload → adapter → insert (duplicate-phone-aware, same as before) → classify.	`importAndClassifyClients`
5. AI Client Analysis	Clickable per-type count cards with drill-down; "Assign Business Type" bulk flow for Unclassified clients (multi-select + existing-or-new-type picker) — no spreadsheet editing ever required.	`assignBusinessTypeAction`
6. Configure Required Documents	Per business type: existing requirements (removable), suggested-but-not-added quick-adds, free-text custom document add.	reused `addRequirement` / `removeRequirement`
7. Reminder Rules	Per business type: a "customize for this type" toggle + working days/hours/reminder interval/inactivity timeout, prefilled with the effective (override ?? org default) values.	`updateServiceScheduleOverrides`
8. Summary	Read-only recap, always derived live from real counts (no separate "summary state" to go stale).	—

9. Completion	Best-effort activates automation if both integrations were connected (BR-001 gate, unchanged logic), marks onboarding complete, redirects to Dashboard.	Epic 2's existing `finishOnboarding` (extended, see 15.9)

15.9 Post-Wizard Editability

Every setting the wizard configures remains editable afterward, per explicit requirement:

- **Office name/logo** — new `OfficeInfoForm` component (`src/components/app/OfficeInfoForm.tsx`) shared by Step 2 and a new section on `/settings` ; both submit through the same `updateOfficeInfo` action.
- **Document requirements per business type** — already fully editable via the pre-existing `/services/[id]` page (a business type *is* that service); no new UI needed.
- **Reminder overrides per business type** — new `ServiceScheduleOverrideCard` component (`src/components/app/ServiceScheduleOverrideCard.tsx`) shared by Step 7 and a new card on `/services/[id]` .
- **Automation on/off** — `activateAutomation` / `deactivateAutomation` (Epic 2, unchanged core logic) now surface as a toggle on `/settings` instead of only inside the old single-page Office Setup — the wizard itself no longer shows this control, only Step 9's best-effort auto-activation.

15.10 Testing Performed

Same real-HTTP, scripted-browser standard as prior epics (Playwright, temporarily installed for the run, removed afterward). A single continuous pass: register a fresh organization → walk all 9 steps (real CSV fixture with a deliberate mix of classifiable and unclassifiable Hebrew business names) → verify AI analysis counts, bulk-reclassification, custom document addition, and reminder-override saving are all *visibly* reflected without a manual reload → finish → confirm `/services` lists the five business-type-backed services with their configured requirements → confirm the new Reminder Overrides and Settings sections render → log out and back in as the same (now-onboarded) org → confirm it lands straight on `/dashboard` , not back in the wizard → log in as a pre-Epic-3 account → confirm unaffected. `npm run build` and `npm run lint` both clean.

15.11 Known Limitations & Next Steps (Epic 3)

Item	Why / Next step
Classification heuristics are a small, fixed rule set	Sufficient for a mocked pilot; a real LLM classifier (already interface-compatible) would handle far more name patterns and languages. See 15.5.
Native .xlsx/.xls parsing not implemented	Same documented security tradeoff as M4 — the adapter (15.6) is ready for a trustworthy parser whenever one becomes available.
Logo storage is a data: URL, not a file host	Fine at the size this is capped to (client-side resized before upload); a real asset host would be needed for larger images or multiple assets per organization.

Custom Business Types have no suggested requirement template	Only the five starter types ship with a pre-defined checklist (15.3) — an office-created custom type starts empty, filled entirely via "Add Custom Document."

Epic 3 addendum — generated from the actual repository state (git history, schema, and a live scripted end-to-end test pass) at completion of the Premium First-Time Onboarding Wizard epic. All test results and both bug findings described were executed and observed, not projected.

ADDENDUM

Centro Implementation Report

Production-Ready Client Import: Native Excel, Hebrew Business-Type Recognition & Localization

This addendum extends the Centro Implementation Report and its Epic 2/3 addenda with a follow-up hardening pass on the onboarding wizard's client-import step, prompted by real testing against an actual Israeli accounting office's file.

Prepared for	noammaarachot@gmail.com
Addendum covers	Native .xlsx support, deterministic Hebrew business-type mapping, full Hebrew localization of the onboarding wizard
Applies on top of	M1–M13 report + Epic 2 & Epic 3 addenda (bound before this section)

16. Client Import Hardening — Native Excel, Business-Type Recognition, Localization

16.1 Executive Summary

Epic 3 shipped a working CSV-only import with a rejected-with-a-friendly-message stub for .xlsx, and English canonical names for the five starter Business Types. Real testing surfaced three concrete gaps: offices work in Excel, not CSV, by default; the classifier's synonym list didn't cover the actual Hebrew terminology Israeli accounting files use; and the wizard showed those English internal names directly to Hebrew-speaking users. All three are fixed below, plus a fourth, related fix: the classifier function is now clearly structured as a deterministic match first, with the (currently unconfigured) AI path explicitly gated to run only after deterministic matching has failed — matching the intended architecture, not just its label.

NEW DEPENDENCY	NEW VULNERABILITIES ADDED	BUSINESS-TYPE SYNONYMS	AUTOMATED TESTS ADDED
exceljs (read-only)	**0 (verified)**	**30+**	**28**

TEST RUNNER
vitest (new)

16.2 Native Excel (.xlsx) Support

Library selection process (documented per the explicit requirement to research rather than assume): the SheetJS `xlsx` package — the option historically rejected in M4/Epic 3 — remains at npm version 0.18.5, years stale, because SheetJS stopped publishing patched releases to the public registry. `exceljs` was evaluated instead: installed and checked with `npm audit` before being accepted, not after. The install itself added zero new advisories; a subsequent full `npm install` surfaced one transitive finding — exceljs pins `uuid@8.3.2`, and that major version has a published moderate advisory (GHSA-w5hq-g745-h8pq, a missing bounds check reachable only when a caller supplies a custom buffer to uuid's v3/v5/v6 functions — not a pattern this codebase's read-only usage triggers, but not accepted regardless). Fixed with an npm `overrides` entry in `package.json` pinning `uuid` to `^11.1.1` across the whole tree; verified via `npm audit` and a read/write round-trip smoke test afterward. The project's final audit count (6 moderate findings, all pre-existing in `drizzle-kit`'s esbuild toolchain and Next's postcss, unrelated to this change) did not increase.

Usage is deliberately narrow: only `Workbook.xlsx.load()` and worksheet/row iteration are called (`src/lib/import/xlsxParser.ts`) — exceljs's write and streaming APIs are never invoked anywhere in this codebase, keeping the exposed surface to exactly "parse an untrusted spreadsheet into strings," regardless of what else the package can do.

Pipeline reuse: `src/lib/csv.ts` was split into `parseCsv()` (format-specific: turns CSV text into a raw `string[][]` grid) and the new `mapRowsToClientRows()` (format-agnostic: turns that grid into `{ name, phone,`

`email, notes, businessType }` via the Hebrew/English header-alias table). `xlsxParser.ts` produces the same `string[][]` shape from a worksheet, so it calls the exact same `mapRowsToClientRows()` — column detection, required-field validation, and business-type-column recognition are implemented exactly once for both formats, not duplicated. `src/lib/import/clientImportAdapter.ts` dispatches by file extension; legacy binary `.xls` (a structurally unrelated, pre-2007 format with no comparably safe actively-maintained parser available) is explicitly out of scope and rejected with a message naming the two formats that are supported, rather than a raw parse error.

16.3 Deterministic Hebrew Business-Type Recognition

Root cause of "detected all clients but classified none"

The classifier logic itself was already capable of matching terms like **עוסק פטור** — the actual gap was coverage and structure: the synonym list was small (only the full legal phrases, not the shorthand terms real files use, e.g. **מורשה** alone rather than always **עוסק מורשה**), and it lived as ad-hoc regex inside the classifier rather than as an explicit, reviewable dictionary. Rebuilt as `BUSINESS_TYPE_SYNONYMS` in `src/lib/ai/businessTypeClassifier.ts` — an explicit canonical-name-to-synonym-list map, matched by normalized substring containment (longest synonym first, so a specific phrase is preferred over a word that's also a substring of it), covering every example given plus the common Hebrew/English variants around it.

Canonical (Hebrew, as stored)	Recognized synonyms (subset)
חברה בע"מ	plus Ltd/Inc/Corp/Company — בע"מ, בעמ, חברה
עוסק מורשה	plus sole proprietor/self-employed — עוסק/ת מורשה, מורשה
עוסק פטור	plus exempt/exempt business — עוסק/ת פטור, פטור
עמותה	plus nonprofit/NGO/amuta — עמותת
שכר בלבד	plus payroll/payroll only — שכר

Priority order, made explicit in code and its doc comment (`classifyClientBusinessType`):

1. Deterministic dictionary match against an explicit business-type column, when the import file provides one (real, office-supplied data) — confidence 1.0.
2. **AI fallback** — a dedicated `classifyViaAI()` function, called only if step 1 found nothing. No LLM/OCR provider is configured for this pilot (same mock-first decision as `documentClassifier.ts`), so this is currently a documented no-op returning `null` — not a fake guess — and wiring in a real provider later touches only this one function, which by construction cannot run before deterministic matching has had its chance.
3. Deterministic dictionary match against the client's own *name* (some offices embed the type there instead of a separate column) — lower confidence, last resort.

4. Unclassified — always available for manual assignment in Step 5, never silently guessed.

The business-type column's own header detection was also broadened (`src/lib/csv.ts` 's `HEADER_ALIASES`) to recognize more realistic real-world column names: **סוג עסק, סיווג, סוג ישות, ישות משפטית, מעמד, מעמד משפטי, סוג התאגדות, סטטוס עסק**, plus the English equivalents.

16.4 Full Hebrew Localization

The five starter Business Types' canonical *stored* names (`src/lib/businessTypes.ts` 's `STARTER_BUSINESS_TYPES`) were changed from English to Hebrew directly — **חברה בע"מ, עוסק מורשה, עוסק פטור, עמותה, שכר בלבד** — rather than adding a separate display-translation layer. Because every wizard step (5, 6, 7, 8) and the post-wizard `/services` page already render this field dynamically rather than hardcoding labels, this single seed-data change is what makes the whole experience Hebrew, with no risk of a translation layer drifting out of sync with the underlying data. The suggested document-requirement names seeded per type (e.g. "Expense Invoices") were translated to their standard Hebrew accounting terms (**חשבוניות הוצאה**, etc.) for the same reason — those are also rendered directly in Step 6. Everything internal that isn't user-facing (event-type strings, code identifiers, comments) intentionally stayed in English, consistent with the rest of this codebase.

Note on existing data: organizations that completed onboarding before this change have business types already seeded with the old English names — this only affects the local pilot/dev database (no production tenants exist yet), and every name remains fully editable per-organization from Step 6 or `/services` regardless.

16.5 Post-Import UX

Step 5's top banner was restructured into an explicit three-number success summary — clients imported, classified automatically, and requiring manual classification — replacing the single "N clients found" line. This sits directly above the existing per-type drill-down cards and the "Assign Business Type" bulk-classification flow for anything left unclassified, so the full picture (what happened, and what — if anything — still needs a decision) is visible in one place immediately after upload.

16.6 Automated Testing

No test runner existed in this project before this change. Added `vitest` (zero new advisories, minimal config — no DOM/plugin dependency needed since these are unit tests over parsing/matching logic, not component rendering) with `npm run test` . 28 tests across four files: `csv.test.ts` (RFC 4180 edge cases including the Epic-3 בע"מ mid-field-quote fix, Hebrew header aliasing, the required-column error path), `xlsxParser.test.ts` (a real exceljs-generated workbook parsed back, numeric-cell stringification, blank-row skipping, Hebrew round-tripping), `clientImportAdapter.test.ts` (end-to-end dispatch for both real formats plus the .xls/unknown-extension rejection paths), and `businessTypeClassifier.test.ts` (every synonym example from this requirement, the name-fallback path, and the "don't match a type the org never created" guard).

16.7 Files Touched

File	Change
`package.json`	+ `exceljs`, + `vitest` (dev), `overrides.exceljs.uuid`, `test` script
`src/lib/csv.ts`	split into `parseCsv()` + `mapRowsToClientRows()`; broadened business-type header aliases
`src/lib/import/xlsxParser.ts`	**new** — native .xlsx reading
`src/lib/import/clientImportAdapter.ts`	.xlsx now real; .xls/unknown messaging updated
`src/lib/ai/businessTypeClassifier.ts`	rebuilt around an explicit synonym dictionary + priority-ordered, AI-gated-as-fallback matching
`src/lib/businessTypes.ts`	starter type + requirement names translated to Hebrew
`src/app/onboarding/actions.ts`	`await` on the now-async classifier; richer audit-event metadata
`src/app/onboarding/steps/Step5Analysis.tsx`	explicit imported/classified/needs-review success summary
4 new `*.test.ts` files, `vitest.config.ts`	automated test suite

Addendum generated from the actual repository state (git history, a full `npm audit` pass, and the automated + scripted browser test results described above) after this hardening pass. All findings and test results were executed and observed, not projected.

ADDENDUM

Centro Implementation Report

Business-Type Data Migration: Upgrading Pre-Existing Organizations to Hebrew

This addendum documents a bug found during real-user re-testing of the previous (Import Hardening) addendum: organizations seeded before the English→Hebrew business-type rename were never upgraded, so both the wizard's display and client classification silently kept failing for them. Fixed with an automatic data migration, not a fresh-database requirement.

Prepared for	noammaarachot@gmail.com
Addendum covers	Root cause analysis, the data migration, and end-to-end re-verification
Applies on top of	M1–M13 report + Epic 2, Epic 3 & Import Hardening addenda (bound before this section)

17. Business-Type Data Migration — Upgrading Existing Organizations

17.1 Bug Report

The previous addendum's own "Note on existing data" flagged, but did not act on, a gap: it stated the English→Hebrew rename "only affects the local pilot/dev database" and every name "remains fully editable." Real re-testing proved that caveat insufficient — the reporter's own test organization still showed the old English labels (Limited Company, Sole Proprietor, Exempt Business, Payroll Only, Nonprofit) throughout the onboarding UI, and a 20-client import file with a populated Business Type column produced **zero** automatic classifications — every client fell through to "requires manual classification," despite the classifier logic itself being correct and already covered by 28 passing unit tests.

17.2 Root Cause

One root cause, two visible symptoms

`seedStarterBusinessTypes()` is deliberately idempotent-by-name: it only creates starter types an organization doesn't already have, so that a firm which has already customized its checklist is never silently overwritten. That guarantee has a corollary its author didn't account for: when the starter list's canonical names changed from English to Hebrew, organizations that had already been seeded under the old names were *never revisited* — the idempotency check saw "this org already has a type named 'Limited Company'" and correctly left it alone, because at the code level nothing distinguishes "the user renamed this on purpose" from "this is stale seed data."

That single stale row set explains both symptoms:

1. **English UI** — Steps 5–7 and `/services` render `business_types.name` / `services.name` directly; a stale org's rows were still English.
2. **Zero classification** — `classifyClientBusinessType()` resolves an explicit "business type" column value (e.g. חברה בע"מ) to its Hebrew canonical name via `BUSINESS_TYPE_SYNONYMS`, then calls `findCandidateByName()` against the organization's actual `business_types` rows. For a stale org those rows were still named `"Limited Company"` etc. — an exact-string lookup for the Hebrew name found no candidate, so *every* classification attempt silently fell through to Unclassified, regardless of a correctly populated source column.

Confirmed empirically before writing any fix, not assumed: a direct query of every local organization's `business_types` rows showed 2 of 6 still on the old English names — including an organization named רז רואי חשבון (a genuine Hebrew accounting-firm name), matching the reporter's own account exactly.

17.3 Fix: An Automatic Upgrade Migration

Per the explicit requirement — *upgrade existing installations automatically instead of requiring a fresh database* — the fix is a proper, hand-authored Drizzle data migration (`drizzle/0009_hebrew_business_type_names.sql`), generated via `drizzle-kit generate --custom` so it is correctly registered in `drizzle/meta/_journal.json` and runs through the exact same `npm run db:migrate` pipeline as every other migration — not a one-off script that would only help this one machine.

The migration, scoped conservatively in three layers:

1. Renames `business_types.name` from each of the five original English defaults to its Hebrew canonical equivalent, matched by exact old name — an organization that already renamed a type to something else entirely is left untouched.
2. Renames the matching `services.name` row (the backing service is what `/services` and Collection Requests actually display), same exact-match scoping.
3. Renames the standard document-requirement labels those five services were originally seeded with (e.g. "Expense Invoices" → **חשבוניות הוצאה**), scoped by `service_id IN (SELECT id FROM services WHERE name IN (<the Hebrew names just written above>))` — so a requirement a user manually added to some unrelated custom service under a coincidentally matching English label is never touched.

PGlite constraint discovered while authoring this migration

PGlite (and the underlying Postgres protocol) cannot prepare more than one SQL command per statement. Drizzle's migrator splits a migration file into separate prepared statements on `--> statement-breakpoint` markers, so every individual `UPDATE` needs its *own* marker — batching several statements under one shared marker throws `error: cannot insert multiple commands into a prepared statement` (Postgres 42601). The final migration has 15 separate breakpoints, one per statement.

17.4 Verification

Confirmed via direct DB inspection, then a full scripted re-run of the wizard

After applying the migration, all 6 local organizations — including the previously-stale **רז רואי חשבון** — were re-queried directly: every `business_types` , `services` , and `service_document_requirements` row is now correctly Hebrew.

A full, fresh browser-driven pass of the onboarding wizard was then run end-to-end (registration → Step 1 through Step 5) against a new organization, importing a 20-row CSV fixture deliberately built to exercise every synonym-matching path: 5 rows using the exact Hebrew canonical terms, 5 using English/abbreviated synonyms (Ltd, Sole Proprietor, Nonprofit, ...), 5 using short Hebrew shorthand (**amuta, payroll ,פטור ,מורשה ,בע"מ**), and 5 with an *empty* business-type column to exercise the name-based fallback and the final Unclassified path.

Check	Result

English business-type strings anywhere in the Step 5 UI	0 (none found)
Expected Hebrew canonical names rendered	All 5 present (חברה בע"מ, עוסק מורשה, עוסק פטור, עמותה, שכר בלבד)
Clients imported	20
Auto-classified	19 (4 / 4 / 4 / 4 / 3 across the five types)
Requiring manual classification	1 — a name with no recognizable business-type signal at all (correctly deferred to manual assignment, never guessed)

This matches the deterministic classifier's documented behavior exactly: every row with a recognizable signal (explicit column or embedded in the name) classified correctly at confidence 1.0 or 0.6 respectively; the one row with neither correctly fell through to Unclassified rather than being silently mis-guessed.

Full regression pass after the fix: `npm run build` (all 17 routes compiled), `npm run lint` (clean), and `npm run test` (28/28 passing, unchanged) — the migration is data-only and touches no schema or application code, so no existing test coverage was expected to move, and none did.

17.5 Files Touched

File	Change
`drizzle/0009_hebrew_business_type_names.sql`	**new** — hand-authored data-only migration upgrading pre-existing organizations
`drizzle/meta/_journal.json`	registers migration 0009
`drizzle/meta/0009_snapshot.json`	**new** — unchanged schema snapshot (no schema diff, data-only migration)

Addendum generated from the actual repository and database state — a direct pre/post query of every local organization's business-type data, plus a real browser-driven run of the onboarding wizard through Step 5 with a purpose-built 20-row CSV fixture — after this fix. All findings and test results were executed and observed, not projected.

ADDENDUM

Centro Implementation Report

Content-Based Column Inference — Understanding Any Spreadsheet Structure

This addendum replaces header-alias matching as the client-import pipeline's primary mechanism with a genuine, content-based spreadsheet-analysis layer. The previous approach could only recognize a column it already had a name for; this one infers what every column means from the values themselves, works for CSV and XLSX identically, and asks a human to confirm only when the evidence is genuinely ambiguous.

Prepared for	noammaarachot@gmail.com
Addendum covers	The column-inference engine, its scoring model, the mapping-confirmation UI, and end-to-end verification
Applies on top of	M1–M13 report + Epic 2, Epic 3, Import Hardening & Business-Type Migration addenda

18. Content-Based Column Inference

18.1 Problem

Every import pipeline up to this point — including the "hardened" one from the previous addendum — still located the client-name, phone, email, and business-type columns by matching header text against a fixed alias list (`HEADER_ALIASES` in `src/lib/csv.ts`). That works only for files whose headers happen to be one of the strings on the list. A real accounting office's export can rename, reorder, drop, or translate those headers, omit a header row entirely, or add unrelated columns — none of which a fixed alias list can anticipate. The requirement was explicit: Centro must understand a spreadsheet's structure from its data, not from having seen that exact header before, for *any* office's file, not a narrow fix for one test case.

18.2 Architecture

`src/lib/import/columnAnalyzer.ts` is a new, standalone module — no dependency on CSV or XLSX parsing — that takes the plain `string[][]` grid either format parser already produces and scores every column against every candidate role (name, phone, email, business type, notes) using three independent signal families, exactly as specified:

Signal	What it measures	Max weight
Header text	Exact or partial match against a known alias list, in Hebrew and English	20–25% (never the deciding signal alone)
Value pattern	Fraction of non-empty cells matching the role's shape — an Israeli phone-number regex tolerant of separators/country-code/lost-leading-zero, an email regex, the existing Hebrew/English business-type synonym dictionary, or a lenient "looks like text, not a phone or email" name check	40–75% (dominant signal for every role)
Structure	Cardinality relative to row count — a client-identity column (name/phone/email) should be close to one distinct value per row; a categorical column (business type) should repeat a small set of values across many rows	5–35%

The structure signal is what keeps a company-name column that happens to contain **בע"מ** from ever being mistaken for the business-type column — its values are still almost all unique, while a genuine business-type column repeats. Assignment is sequential and greedy — phone first (the least ambiguous pattern, a digit-string regex), then name, then business type (accepted only above its own confidence bar, otherwise left unmapped — requirement 9's "no business-type column found" path), then email, then notes — removing each claimed column before the next role is scored, so a column already confidently claimed for one role can never simultaneously mislead the confidence check for another.

Confidence and the human fallback: after every role is resolved, each required role's assigned column is checked against every column that ended up genuinely unclaimed by any role — if a close, unclaimed rival

exists, or the absolute score is weak, confidence is `"low"` and the wizard shows a short mapping-confirmation screen instead of guessing (requirement 8). An `inferColumnRolesViaAI` stub — the same documented "mock-first" pattern as `classifyViaAI` in the existing per-row classifier — sits between deterministic scoring and the human fallback, exactly matching the required priority order (deterministic first, AI second, human last), even though no LLM provider is configured for this pilot.

Both formats, one pipeline: `clientImportAdapter.ts` now exposes `parseClientImportFileToGrid()`, returning the same grid shape for CSV and XLSX; column understanding is implemented exactly once and used identically by both.

18.3 The Mapping-Confirmation Screen

When confidence is low, `importAndClassifyClients` (Step 4's action) returns a `needsMapping` payload instead of importing anything — the parsed grid, detected headers (synthesized as **"עמודה N"** when there is no header row), a sample of the data, and the analyzer's best guess per role. `Step4Import.tsx` renders this as a short form: one labeled dropdown per role ("**לדעתנו זו עמודת שם הלקוח**" / phone / email / business type), pre-filled with the suggestion, listing every column with a live sample value, plus a small preview table of the first rows. Submitting calls a new action, `confirmImportMapping`, which finishes the exact same import+classify logic (now factored into a shared `importParsedRows` helper) using the human-confirmed mapping — the original file never needs to be re-uploaded, since the parsed grid round-trips through the confirmation form as JSON.

18.4 Analysis Summary in the UI

Requirement 14: Step 5 now shows, above the existing imported/classified/needs-review banner, a row of badges naming exactly which column Centro used for each detected role and whether the match was confident or came from a manual confirmation. This is sourced from a new audit event, `clients.import_analyzed` (recorded by `importParsedRows` alongside the existing `clients.imported` / `clients.classified` events), read back by a new `getLatestAuditEventByType()` query — reusing the existing audit-log infrastructure rather than inventing a parallel persistence mechanism, since Step 5 is server-rendered after Step 4's redirect and has no other way to see that action's return value.

18.5 Row-Level Safety (Requirement 10)

Column-level detection and row-level classification are deliberately separate concerns. Once the business-type *column* is found — even from a mix of valid and noisy values (city names, free-text notes, stray numbers, blanks) — every individual row still goes through the existing, unchanged `classifyClientBusinessType()` (src/lib/ai/businessTypeClassifier.ts), which only ever assigns a type it can actually match and leaves anything else `null` (Unclassified, routed to Step 5's manual bulk-assignment flow). A noisy value degrading the column's detection score slightly never results in a wrong classification for that specific row — it results, correctly, in that one row needing manual review while every valid row around it still classifies automatically.

18.6 Automated Testing

`src/lib/import/columnAnalyzer.test.ts` — 18 new tests, covering every case listed in the requirement: detector-level tests for phone numbers across real-world formats (dashes, spaces, parens, international prefix, numeric-stored values with a lost leading zero) and for the full Hebrew/English business-type vocabulary; column-level tests for entirely generic/unknown headers, a missing header row, mixed valid and invalid business-type values (verifying noisy rows don't corrupt column detection while still passing through unclassified, never forced), company names instead of person names (verifying the structural cardinality signal keeps this from colliding with business-type detection), extra unrelated columns, ragged rows (a data row shorter than the header), duplicate phone numbers, and the low-confidence path itself (including a direct test of the manual-mapping build path the confirmation screen relies on). Combined with the existing suite, the project's total is now 46 passing tests across 5 files (up from 28/4).

`src/lib/csv.ts`'s `mapRowsToClientRows()` — now a thin convenience wrapper over the new analyzer — was verified to keep every one of its existing tests passing unchanged, along with the existing XLSX round-trip and adapter tests, so this is a genuine drop-in upgrade of the detection layer, not a breaking change to any existing contract.

18.7 End-to-End Verification

Three real browser-driven scenarios, both file formats

Scenario	File	Result
Clean, well-labeled file (regression check)	CSV, standard headers	Unchanged behavior — straight to Step 5, high confidence, 6/6 classified, zero English leaked
Entirely generic/unknown headers	XLSX, headers **"פרטים 1"–"פרטים 4"**	Resolved automatically from content alone — high confidence, no confirmation screen, 11/11 imported and classified correctly
Genuinely ambiguous structure	XLSX, two equally phone-shaped columns (mobile + fax), no header hints	Correctly triggered the mapping-confirmation screen listing both candidate columns; import completed successfully (4/4 classified) after confirming the suggested mapping

All three scenarios were run as fresh registrations through the real onboarding wizard in a headless browser (not a unit-test simulation), confirming the analyzer, the confirmation UI, and the shared import+classify pipeline all work together correctly for both file formats.

18.8 Files Touched

File	Change
`src/lib/import/columnAnalyzer.ts`	**new** — the content-based inference engine

`src/lib/import/columnAnalyzer.test.ts`	**new** — 18 tests
`src/lib/csv.ts`	`mapRowsToClientRows()` rebuilt on the analyzer; new `buildClientRowsFromMapping()` pure mapping-application helper
`src/lib/import/clientImportAdapter.ts`	new `parseClientImportFileToGrid()` raw-grid entry point shared by both formats
`src/lib/ai/businessTypeClassifier.ts`	`matchSynonym` exported for reuse by the column analyzer
`src/app/onboarding/actions.ts`	`importAndClassifyClients` now analyzer-driven with a `needsMapping` result; new `confirmImportMapping` action; shared `importParsedRows` helper; new `clients.import_analyzed` audit event
`src/app/onboarding/steps/Step4Import.tsx`	new mapping-confirmation sub-view with per-role dropdowns and a data preview
`src/app/onboarding/steps/Step5Analysis.tsx`	new detected-columns/confidence summary banner
`src/lib/data/auditLog.ts`	new `getLatestAuditEventByType()` query

Addendum generated from the actual repository state — 46/46 automated tests, a clean production build and lint pass, and three real browser-driven end-to-end scenarios across both CSV and XLSX — after this change. All findings and test results were executed and observed, not projected.

EPIC 4

Centro Implementation Report

The Smart Import Engine — A Document-Understanding Redesign of Client Import

A full architectural rebuild of client import, not an incremental fix: spreadsheet structure understanding, content-only column inference extended to eight field types, multi-layer business-type classification with numeric confidence and human-readable reasoning, and a per-organization learning mechanism — so Centro adapts to any real accounting office's file, never the other way around.

Prepared for	noammaarachot@gmail.com
Addendum covers	Spreadsheet understanding, column inference, the classification pipeline, confidence scoring, the learning mechanism, and why the design scales
Applies on top of	M1–M13 report + Epic 2, Epic 3, Import Hardening, Business-Type Migration & Column-Inference addenda

19. The Smart Import Engine (Epic 4)

19.1 Why a Redesign, Not a Fix

The previous addendum's column-inference layer solved "don't trust the header text" for four fields. It did not solve everything a real accounting office's export actually looks like: a title row above the table, a "total" row below it, several worksheets in one workbook, columns in an arbitrary order, fields Centro had never modeled (city, company registration number, tax ID), business-type values missing outright, or an office's own shorthand no dictionary could know in advance. This epic treats import as a document- understanding problem end to end — from "what is this file's actual structure" through "what does each column mean" to "what does this specific office call this business type" — rather than patching the previous column-matcher one gap at a time.

NEW MODULES	COLUMN ROLES RECOGNIZED	NEW DB OBJECTS	CLASSIFICATION LAYERS
2 (spreadsheetStructure, +xlsxParser rewrite)	**8 (was 5)**	**1 table, 3 columns**	**4 (learned → dictionary → context → AI)**

AUTOMATED TESTS
67 (+21 this epic)

19.2 STEP 1 — Spreadsheet Understanding

Before any column is scored, the file is understood as a document, not just a grid:

- **Multiple worksheets** (`xlsxParser.ts` 's new `parseXlsxWorkbook`) — every sheet is read, and the one with the most non-empty rows is used as the client table, so a workbook with a cover/instructions sheet never accidentally imports the wrong one. Every sheet name found, and which was used, is reported in the wizard's understanding summary.
- **Merged cells** — verified directly against ExcelJS's actual behavior rather than assumed: a merged range's value is already resolved onto every cell in that range by the time the workbook finishes loading (confirmed with a throwaway script before writing any handling code), so a business-type label merged across several client rows is never lost. The engine detects and reports that merges were present; no extra propagation code was needed once that was verified.
- **Hidden columns** — detected via each column's `hidden` flag and reported, but never excluded: a hidden column is frequently still real data an office just collapsed for display.
- **Where the table actually starts and ends** (`spreadsheetStructure.ts` , format-agnostic — the same function trims a CSV or an XLSX grid) — the table's real column count is estimated as the most common non-empty-cell-count across all rows; a leading title row or trailing "total: N clients" row won't share that width (or matches a small set of summary keywords) and is trimmed before column analysis ever sees it.

19.3 STEP 2 — Column Detection, Extended

`columnAnalyzer.ts` (from the previous addendum) now scores eight roles instead of five — name, phone, email, business type, and notes as before, plus city, company ID (ח.פ), and tax ID. The three new roles are deliberately **best-effort, never required, never gating confidence**: city is a categorical value matched against a curated Israeli city list; company/tax IDs are structurally identical (both plain 9-digit numbers) and, honestly, cannot be told apart — or fully told apart from a phone number missing its leading zero — from content alone, so header text is weighted far more heavily for these two roles specifically than for any other. That limitation is documented in code rather than glossed over.

A real bug found during verification: shuffled-column tie collision

Testing with a genuinely shuffled, header-less file surfaced a bug the previous addendum's fixtures never exposed: a business-type column made of varied synonym forms ("חברה בע\"מ", "מורשה", "פטור", "עמותה", "Payroll") has five *literally distinct* string values, so it can score identically to a genuine name column on the "name" role (same pattern and cardinality signals). The previous tie-break — whichever column appears first — happened to always pick correctly by coincidence of column order in every earlier test file; a shuffled file with the business-type column positioned before the name column exposed the real flaw: it stole the name role outright.

Fix: ties are now broken by *specificity* — a column's score for a role minus its own best score among every other role. The real name column has a wide gap between "name" and its next-best role; the colliding business-type column doesn't (it also scores well on "business type" itself). This is a genuine algorithmic improvement, not a special case for the one file that exposed it — a regression test using that exact shuffled shape is now part of the suite.

19.4 STEP 3 — Multi-Layer Business-Type Classification

`classifyClientBusinessType()` (src/lib/ai/businessTypeClassifier.ts) now runs a strict four-layer priority chain per client row, never a single cell in isolation:

1. **Learned synonyms** (see 19.6) — this specific organization's own past correction, checked first because it is the most specific signal available.
2. **Global deterministic dictionary** on the explicit business-type column, when present — expanded this epic to cover every documented shorthand (ע. מורשה, ע. פטור, מלכ"ר, חל"צ, and more) alongside the existing full forms.
3. **Context inference** — when the explicit column is empty or unrecognized, the same two dictionaries are checked again against the client's name, then every other non-empty cell in the row (city, notes, an unmapped extra column — everything `columnAnalyzer.ts`'s new `extractRowContextValues()` collects). A company named "אלפא פתרונות בע\"מ" with a blank type column still classifies correctly from its own name; this is still deterministic matching, just against different input, never a guess.
4. **AI fallback** — reached only once every deterministic layer has failed, given the *entire row*, not one cell, and contracted to return a business type with its own confidence and a reason. No LLM provider is configured for this pilot, so this remains a documented no-op stub (the same "mock-first" pattern as every other AI seam in this codebase) — never a fake guess, and wiring one in later touches only this function.

Every result carries a human-readable `reason` string alongside its confidence, for the audit trail and future UI surfacing.

19.5 STEP 4 — Confidence Scoring

Band	Behavior	Typical source
95–100%	Auto-classified silently	Explicit column, exact/synonym dictionary match, or a learned synonym (all 99%)
70–94%	Auto-classified, flagged "suggested" in the UI	Context inference (85%) — the client's name or another cell, not a dedicated column
<70%	Never applied — left Unclassified, routed to manual bulk assignment	No deterministic or AI signal found anywhere in the row

`clients.businessTypeConfidence` (new column) persists whichever score applied, so Step 5 can distinguish "certain" from "suggested" per client, not just aggregate counts.

19.6 STEP 7 — Learning From Corrections

Scoped strictly per organization — verified with two separate import sessions

A new `business_type_learned_synonyms` table (organization ID, business type ID, normalized source text, unique per org+text) is written the moment a human manually assigns a Business Type to a client whose import row carried unrecognized raw text (`clients.importedBusinessTypeText` , also new). Every subsequent import for that same organization checks this table first — before the global dictionary — so the office's own shorthand is recognized immediately, with no global retraining and zero effect on any other organization's data.

Verified end to end, not just unit-tested: a client whose business-type column read the made-up term "עוסק-מ" (deliberately absent from any dictionary) imported as Unclassified in one browser session; manually assigning it to עוסק מורשה in Step 5 recorded the synonym; a second, independent browser session — a fresh login, not a continuation — then imported two new clients using the exact same term and both classified automatically, with zero manual review needed.

19.7 STEP 6 — Stable Internal Model

This epic closes out a real incident from two addenda ago: renaming the five starter Business Types' display label from English to Hebrew silently broke every organization's classification and UI, because matching depended on the exact display string. `business_types` now has a `canonical_key` column — a stable, renaming-proof identity (`limited_company` , `authorized_dealer` , `exempt_dealer` , `nonprofit` , `payroll_only`) for the five standard types, backfilled onto every existing organization's rows in the same migration. The classifier now matches a synonym to a canonical key, then looks up the organization's row by that key — `business_types.name` is purely a display label from here on, freely editable per-organization exactly as before, with no way for a rename to ever again silently break matching. Org-created custom types have no global canonical concept and are still matched by name, unaffected.

19.8 Why This Scales

- **Deterministic-first, always.** Every layer — column detection, business-type matching, learned synonyms — is content-based pattern matching against explicit, reviewable dictionaries, not a model call. The (currently unconfigured) AI fallback is structurally the last resort in both the column-detection and classification pipelines, never the first thing tried, so accuracy for the overwhelming majority of real files doesn't depend on an LLM being available, fast, or cheap.
- **No per-office code.** Nothing in this epic references a specific file, header, or office. Every fixture used for verification was deliberately constructed to be structurally different from the others (shuffled columns, no headers, multiple sheets, title/footer rows, missing values) specifically to prove the engine generalizes rather than having been tuned to one example.
- **Learning is additive and isolated.** Each organization's learned synonyms only ever improve that organization's own future imports; there is no shared model, no cross-tenant leakage, and no retraining step — a new row in one small table is the entire mechanism.
- **Confidence gates automation, not accuracy claims.** The 95/70 bands mean a low-signal row is always routed to a human rather than silently guessed — the system's accuracy on real files improves the false-negative rate (fewer wrongly-Unclassified rows), never the false-positive rate (a wrongly-classified row), which is the property that actually matters for a system managing real client data.

19.9 Verification

67 automated tests (21 new) + 3 real browser-driven scenarios

New/extended suites: `columnAnalyzer.test.ts` (bonus roles, row- context extraction, the shuffled-column regression), `spreadsheetStructure.test.ts` (new — table-bounds detection), `xlsxParser.test.ts` (multi-sheet/ merge/hidden-column metadata), `clientImportAdapter.test.ts` (STEP 1 structure detection end to end), and a full rewrite of `businessTypeClassifier.test.ts` for the new row-context signature, canonical keys, confidence bands, and learned synonyms.

Scenario	Covers	Result
CSV with a title row, a footer summary row, missing business-type values, and one not-yet-learned shorthand term	STEP 1 table bounds, STEP 3 context inference, confidence tiers, the unclassified path	6 imported (title/blank/footer correctly excluded), 3 auto-classified (99%), 2 suggested via context inference (85%), 1 correctly left for manual review
XLSX with a cover sheet + data sheet, no header row, shuffled column order, plus city and company-ID columns	STEP 1 multi-sheet selection, STEP 2 header-less + shuffled detection, bonus roles	Correct sheet selected automatically and reported; all 6 roles (including city and company ID) detected correctly despite the shuffle; 5/5 classified at 99%
Two independent browser sessions against the same organization, days apart in real usage terms	STEP 7 learning mechanism, org-scoping, persistence	A manually corrected term was recognized automatically on the next import with zero manual review — proven across a genuinely separate login session, not just in-memory state

19.10 Files Touched

File	Change
`src/db/schema.ts`	+ `business_types.canonical_key`, + `clients.businessTypeConfidence` / `importedBusinessTypeText`, new `business_type_learned_synonyms` table
`drizzle/0010_tense_puma.sql`	**new** — schema + canonical-key backfill migration
`src/lib/import/spreadsheetStructure.ts`	**new** — format-agnostic table-bounds detection
`src/lib/import/xlsxParser.ts`	rebuilt around `parseXlsxWorkbook()` — multi-sheet selection, merge/hidden-column reporting
`src/lib/import/columnAnalyzer.ts`	+city/companyId/taxId roles, row-context extraction, specificity tie-break fix
`src/lib/import/clientImportAdapter.ts`	new `analyzeImportFileStructure()` STEP-1 entry point
`src/lib/csv.ts`	`CsvClientRow.otherValues` for row-context classification
`src/lib/ai/businessTypeClassifier.ts`	rebuilt — canonical-key matching, expanded synonyms, 4-layer pipeline, 0-100 confidence, reasons
`src/lib/businessTypes.ts`	canonical keys on starters, learned-synonym read/write, learning hook in `assignClientsToBusinessType`
`src/app/onboarding/actions.ts`	full import-pipeline rewrite around the new structure/classification layers
`src/app/onboarding/steps/Step4Import.tsx` / `Step5Analysis.tsx`	structure metadata round-trip; "Centro understood your file" checkmark summary with confidence tiers
6 test files (2 new)	67 tests total (+21)

Addendum generated from the actual repository state — 67/67 automated tests, a clean production build and lint pass, and three real browser-driven end-to-end scenarios spanning CSV and XLSX, headers and no headers, ordered and shuffled columns, and two independent login sessions proving the learning mechanism — after this redesign. All findings and test results were executed and observed, not projected, including the shuffled-column bug found and fixed during this verification pass.

EPIC 5 — KICKOFF

Centro Implementation Report

Architecture Alignment Roadmap & Milestone 1: Manual-Entry Classification Parity

A new phase begins: aligning the codebase against a formal Architecture & Philosophy document for the first time. This addendum covers the planning process that produced the 7-milestone roadmap, an architectural correction made during planning itself, and the first completed milestone.

Prepared for	noammaarachot@gmail.com
Addendum covers	Architecture review process, roadmap approval, Milestone 1 implementation and verification
Applies on top of	M1–M13 report + Epic 2, 3, 4 addenda

20. Architecture Alignment Roadmap — Planning & Milestone 1

20.1 Context

The user supplied a formal *Centro Architecture & Philosophy* document (8 chapters, since revised to a live in-repo Markdown source — `ARCHITECTURE.md`) as the project's architectural source of truth, and requested a planning-only phase before any code changed: read the document, compare it against the real implementation, and produce a full gap analysis and milestone roadmap.

20.2 Architecture Review & Gap Analysis (summary)

The review was done against the actual codebase — full schema, the collection/ conversation state machine, the scheduler, and all three "AI" classifiers — not from memory. Findings:

- **Already strong:** the Epic 4 business-type classifier is a near-complete reference implementation of the philosophy's Learning/Decision- Hierarchy chapters for one narrow domain — a template to generalize, not rebuild.
- **Partial:** document classification has confidence-gated auto-approval but no "Learned Knowledge" tier; intent classification is logged but never acts; the audit trail is comprehensive but has no severity/exceptions concept beyond the documents queue.
- **Missing entirely:** no "Learning Mode" state on a client; no post-cycle analysis; no client-facing WhatsApp confirmation mechanism (the one working "learn from a correction" example — business-type synonyms — is confirmed by the accountant, never the client); learning is siloed to business-type classification only; clients created outside the onboarding wizard (`/clients/new`) received no classification at all — confirmed by direct inspection of `clients/actions.ts`, which had zero business-type handling before this milestone.

One conflict was surfaced rather than resolved silently: the document's stated Decision Hierarchy order (Business Rules → Learned Knowledge → AI → Human Review) is the reverse of what the working business-type classifier actually does (Learned Knowledge checked first). **Resolved by user decision: keep the implementation, update the document** — Chapter 6 of `ARCHITECTURE.md` now states the hierarchy as actually implemented, with the reasoning documented directly in the chapter (§20.6 below has the exact wording).

20.3 An Architectural Correction Made During Planning, Not After

> **The original Milestone 6 draft violated the product's own boundaries**
>
> The first roadmap draft's Milestone 6 ("Learned Reminder-Timing Suggestions") would have had Centro infer personalized reminder cadence from observed client behavior. The user corrected this immediately and sharply: **Centro is a document collection platform — it learns only which documents to collect, never office policy, timing, or the client's business.** Business hours, working days, reminder cadence, and collection start day are accountant-configured, permanently, with no automated adaptation.

Corrected: Milestone 6 became "Adaptive Document Collection"

Replaced with a milestone that learns exactly one thing — a client's required document list — through the same Observe → Suggest → Confirm → Learn lifecycle, with careful wording discipline added on a second pass (per further user correction): Centro never states a document "is no longer required," only that it *appears* to no longer be part of the regular collection, and always asks before ever removing it from a client's profile.

This correction also revealed that **Milestone 2** (originally "Learning Mode & First-Cycle Analysis") had been scoped to capture cycle-duration and reminder-count facts — timing-adjacent data whose only purpose was feeding the now-removed timing-personalization milestone. Milestone 2 was trimmed accordingly, down to exactly the Learning Mode flag itself, with no timing capture at all — avoiding a component that would have had no legitimate remaining consumer.

The resulting permanent principle — **"Centro learns only which documents should be requested from each client. Centro never learns or modifies office policies such as business hours, working days, reminder cadence, or collection dates."** — is now Chapter 8 of `ARCHITECTURE.md`, explicitly scoped as a constraint on every current and future milestone, not only Milestone 6.

20.4 The Approved Roadmap

#	Milestone	Status
1	Manual-Entry Classification Parity	**Complete (this addendum)**
2	Learning Mode (client-level flag only)	Planned
3	Document Classification Learning	Planned
4	Unified Exceptions & Review Surface	Planned
5	Client-Facing Confirm-via-WhatsApp Loop	Planned
6	Adaptive Document Collection	Planned
7	Pilot Hardening	Planned — final milestone: end-to-end regression across M1–M6, edge-case and org-isolation audit, realistic-data-volume performance check, security review, documentation consolidation, dead-code cleanup, full-app smoke test

20.5 Documentation Process, Formalized

Per explicit user decision: `ARCHITECTURE.md` is now the version-controlled source of truth in the repository (joining `AGENTS.md` / `CLAUDE.md`), rendered to `reports/Centro-Architecture-and-Philosophy.pdf` after every milestone. The Implementation Report continues exactly as before — an append-only PDF history, one addendum per milestone.

20.6 Milestone 1 — Manual-Entry Classification Parity

Goal: a client created through `/clients/new` (not the bulk-import wizard) now goes through the exact same classifier, with the exact same organization-scoped learned synonyms and confidence bands, as an imported client — closing the gap found in §20.2.

What changed:

- `createClient` (`clients/actions.ts`) now runs `classifyClientBusinessType()` against the typed name via context inference (no explicit business-type field exists on this form — the same path a CSV row with an empty business-type column already takes). Applied automatically at ≥70% confidence, exactly like import; left Unclassified below that, never guessed.
- The confidence-band constants (95 / 70) were promoted from a local, duplicated definition inside the onboarding wizard's action file into `businessTypeClassifier.ts` itself and exported — a single source of truth now shared by both the wizard and this new call site.
- New `setClientBusinessType` action plus a "Business Type" card on the client detail page (`/clients/[id]`) — shows the current type with a certain/suggested badge, or an honest "Centro couldn't determine this automatically" message, with a picker (existing type or a new one) always available. A manual choice is stored at 100% confidence — a human decision is definitionally certain, not a guess.

> **Two issues found and fixed during verification (not left for later)**
>
> **Test-script false positive:** the verification script initially expected a clear legal-suffix match (e.g. **אבגד פתרונות בע"מ**) to show as "certain" (95%+). It correctly showed "suggested" (85%) instead — by design, context-inference (matching from a name alone, not an explicit column) is always scored 85, never 95+, which is reserved for explicit-column or learned- synonym matches. The test's expectation was wrong, not the app; fixed in the script.
>
> **Real, small UX bug:** the new "Business Type" card's assignment button and the pre-existing "assign service" button on the same page were both labeled simply **"שיוך"** — ambiguous for automated testing and mildly ambiguous for a human skimming the page too. Renamed to **"שיוך סוג עסק"** for clarity.

20.7 Verification

Scenario	Result
Manually create a client with a clearly classifiable name (company-suffix pattern)	Classified correctly, shown as "suggested" (85%) — never falsely presented as certain

Manually create a client with no classifiable signal at all	Left honestly Unclassified — "Centro couldn't determine this automatically" — no guess
Manually assign a business type from the detail page	Stored at 100% confidence, shown as certain, badge updates immediately

All three scenarios were run as a real, fresh registration through the full onboarding wizard into the live app shell in a headless browser — not simulated. 67/67 existing automated tests continue to pass unchanged (no new pure logic was introduced; this milestone wires already-tested classifier logic into a new call site). Clean production build and lint.

20.8 Files Touched

File	Change
`ARCHITECTURE.md`	**new** — version-controlled source of truth
`reports/Centro-Architecture-and-Philosophy.pdf`	**new** — rendered PDF, regenerated every milestone
`src/lib/ai/businessTypeClassifier.ts`	exported `AUTO_CLASSIFY_CONFIDENCE` / `SUGGESTED_CONFIDENCE`
`src/app/onboarding/actions.ts`	now imports the shared confidence constants instead of a local duplicate
`src/app/(app)/clients/actions.ts`	`createClient` now classifies; new `setClientBusinessType` action
`src/app/(app)/clients/[id]/page.tsx`	new "Business Type" card with confidence badge and manual override

---

Addendum generated from the actual repository state — a real browser-driven verification pass, 67/67 automated tests, and a clean build/lint — after this milestone. All findings were executed and observed, not projected.

MILESTONE 2

Learning Mode

Chapters 1 and 2 of the Architecture & Philosophy document state that every client begins in Learning Mode. This milestone makes that a real, queryable fact rather than prose.

---

21. Learning Mode

21.1 What Was Built

`clients.learningMode` (boolean, default `true`) and `clients.firstCycleCompletedAt` (nullable timestamp) — two new columns, additive migration. `collectionRequestStateMachine.ts`'s `applyTransition()` now calls a small `exitLearningModeIfFirstCycle()` helper whenever a transition successfully reaches `completed`: it flips the flag and stamps the timestamp, scoped by a `WHERE learning_mode = true` clause so it is naturally idempotent — a client's second and every later completed cycle is a silent no-op there, with no extra guard logic needed.

Deliberately excluded, per the Chapter 8 boundary established during roadmap planning: no cycle-duration, reminder-count, or any timing data is captured. This milestone's only purpose is gating Milestone 6's "document appears to no longer be needed" detection — a client with zero completed cycles has no pattern to compare against, so that check must never fire for them.

A small "מצב למידה" (Learning Mode) badge was added to the client detail page header — visible, quiet, informational, consistent with Chapter 7's "invisible but not hidden" automation: an accountant can see the state without it ever demanding attention.

21.2 Verification

A real bug hunt, resolved as a test-script artifact, not an app bug

The first verification pass appeared to fail: after completing a client's first full collection cycle end to end (draft → active → processing → completed, with both starter requirements satisfied), the badge still showed on re-navigation. A direct database query — run standalone, after stopping the dev server per this project's established PGlite single-process discipline — confirmed `learning_mode = false` and `first_cycle_completed_at` correctly set. The failure was Next.js's client-side router cache serving a stale RSC payload for a URL the same browser tab had already visited earlier in the run; re-checking with a fresh page (no navigation history) confirmed correct behavior immediately. Root-caused before concluding, not assumed.

Full scenario, run live in a headless browser: registered a fresh organization, created a client (confirmed Learning Mode badge present), manually assigned it **עוסק פטור** (2 starter requirements), opened a Collection Request, satisfied both requirements, and drove the request through to `completed`. Confirmed the badge was

gone on the next fresh page load. Clean build, lint, and 67/67 existing tests (no new pure logic — this milestone is entirely a state-machine hook, verified live).

21.3 Files Touched

File	Change
`src/db/schema.ts`	+ `clients.learningMode`, + `clients.firstCycleCompletedAt`
`drizzle/0011_lyrical_shiver_man.sql`	**new** — additive migration
`src/lib/collectionRequestStateMachine.ts`	new `exitLearningModeIfFirstCycle()` hook on the completion path
`src/app/(app)/clients/[id]/page.tsx`	Learning Mode badge in the page header

Addendum generated from the actual repository and database state — a real browser- driven end-to-end collection cycle, a direct post-hoc database verification, and a clean build/lint/test pass — after this milestone.

MILESTONE 3

Document Classification Learning

Generalizes Chapter 6's Decision Hierarchy to a second real domain: document classification now learns from a client's own manually-corrected history, not just business-type synonyms.

---

22. Document Classification Learning

22.1 What Was Built

New `document_learned_patterns` table (organization + client scoped, one row per manual correction — filenames vary too much month to month for the exact-text lookup business-type synonyms use, so this is fuzzy token-overlap matching against a client's own history instead). Written from exactly one place: `assignDocumentRequirement` — whenever an employee manually assigns an unmatched document to a requirement, the requirement's stable template identity (`serviceDocumentRequirements.id` , not the frozen per-cycle snapshot row) and the filename are recorded.

`documentClassifier.ts` gains a full 4-layer pipeline — `classifyDocumentWithLearning()` — mirroring `businessTypeClassifier.ts` 's structure: this client's learned patterns first, then the existing deterministic filename-vs-requirement-name heuristic (now formally "the Business Rules layer"), then a new documented AI-fallback stub (no-op, no provider configured — same mock-first pattern as every other AI seam in this codebase), falling through to `needs_review` (Human Review) exactly as before. The plain heuristic remains separately exported and unchanged in behavior for any caller with no learning history to check.

22.2 Verification

12 new unit tests, including a direct proof that the with-learning pipeline produces byte-identical results to the plain heuristic when no learning exists yet (no behavior change for any client until their first correction). 81/81 tests total.

A pre-existing discrepancy found, not introduced — logged for Milestone 7, not fixed here

Writing the first-ever test file for this module surfaced that `isFuzzyDuplicate` 's own doc comment example (**"bank-statement.pdf" vs "bank-statement-copy-2.pdf"**) does not actually clear its own 0.7 token-overlap threshold (it scores 0.5) — a documentation/behavior mismatch that predates this milestone and was never unit-tested before. The logic itself is untouched by this milestone and the direction of the error is the safe one (slightly stricter than intended, so fewer false "duplicate" flags, not more) — logged as a candidate for Milestone 7's edge-case audit rather than changed here, to keep this milestone's diff scoped to what it actually set out to do.

End-to-end proof of the full Observe → Suggest → Confirm → Learn loop

Run live in a headless browser against a fresh organization: uploaded **"טופס_102_ינואר.pdf"** (a filename sharing no tokens with either of the client's two requirement names) — correctly landed in the unmatched- documents queue (Observe/Suggest: the system honestly couldn't match it). An employee manually assigned it (Confirm). A second, later upload for the *same client* — **"טופס_102_פברואר.pdf"**, still sharing no tokens with either requirement's name — was **auto-classified and auto-approved** purely from the one learned pattern (Learn, then applied), with zero manual review required the second time.

22.3 Files Touched

File	Change
`src/db/schema.ts`	new `document_learned_patterns` table
`drizzle/0012_small_titanium_man.sql`	**new** — additive migration
`src/lib/ai/documentClassifier.ts`	rebuilt — `classifyDocumentWithLearning()` 4-layer pipeline, learned-pattern matching, AI-fallback stub
`src/lib/ai/documentClassifier.test.ts`	**new** — 12 tests, first-ever coverage of this module
`src/lib/documentLearning.ts`	**new** — read/write seam for a client's document-learning history
`src/app/(app)/collections/conversationActions.ts`	inbound document classification now goes through the learning pipeline
`src/app/(app)/collections/actions.ts`	`assignDocumentRequirement` now records the learned pattern

Addendum generated from the actual repository state — 81/81 automated tests, a clean build/lint pass, and a real browser-driven end-to-end proof of the learning loop — after this milestone.